

FLOE

Random-Access AEAD for Fast Lightweight Online Encryption

Andrés Fábrega

Greg Rubin

Julia Len

Thomas Ristenpart



Cornell University



The University
of North Carolina
at Chapel Hill

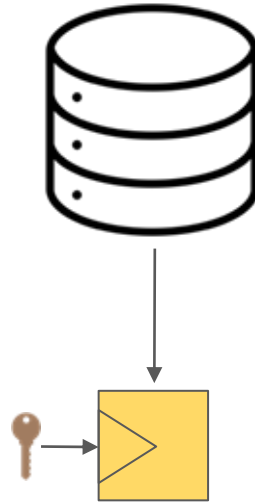


UNIVERSITY OF
TORONTO

We encountered a problem...



- Encryption of **large** files
- Must use **Authenticated Encryption (AES-GCM)**



We encountered a problem...



- Encryption of **large** files
- Must use **Authenticated Encryption (AES-GCM)**



We encountered a problem...



- Encryption of **large** files
- Must use **Authenticated Encryption (AES-GCM)**



AES-GCM, like all Authenticated Encryption (AE), must **examine the entire ciphertext** before returning any plaintext.

Buffering that much data in memory **causes problems**.

We encountered a problem...



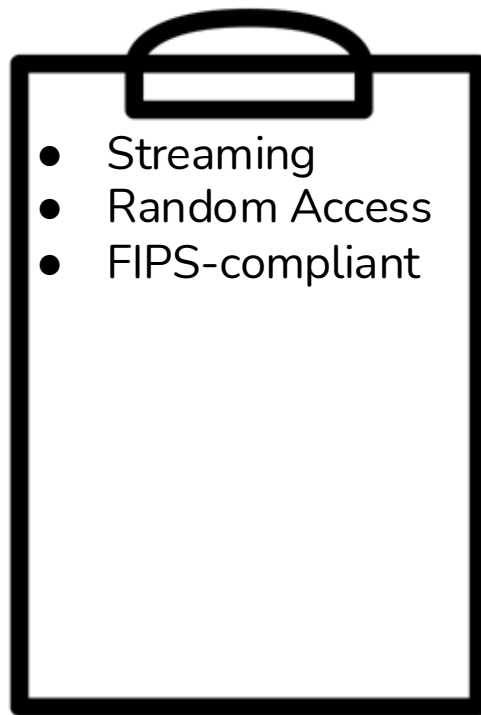
- Encryption of **large** files
- Must use **Authenticated Encryption (AES-GCM)**



AES-GCM, like all Authenticated Encryption (AE), must **examine the entire ciphertext** before returning any plaintext.

Buffering that much data in memory **causes problems**.

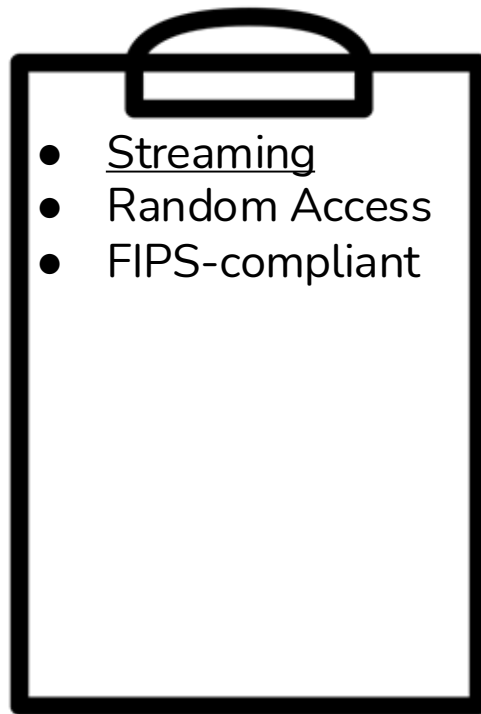
What do we need



What do we need

Streaming:

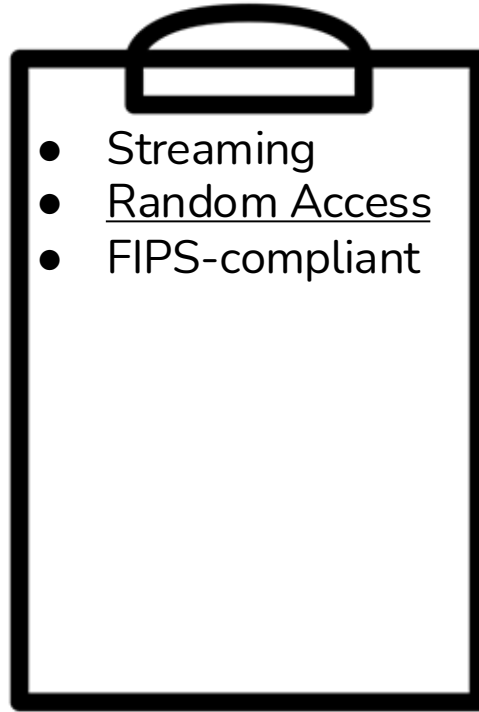
Can encrypt/decrypt large files sequentially with limited memory.



What do we need

Streaming:

Can encrypt/decrypt large files sequentially with limited memory.



Random Access:

Can encrypt/decrypt semi-arbitrary subsections.

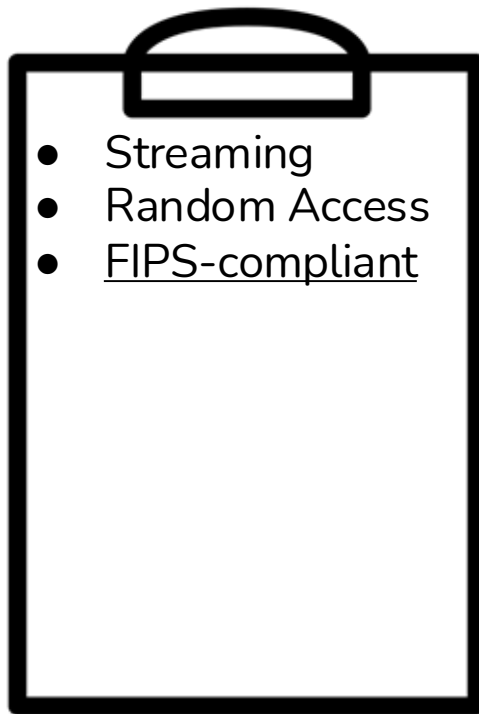
What do we need

Streaming:

Can encrypt/decrypt large files sequentially with limited memory.

FIPS-Compliant:

Only uses approved algorithms (AES-GCM and HKDF_Expand/SP 800-108) in an approved manner (with random IVs).



Random Access:

Can encrypt/decrypt semi-arbitrary subsections.

This is a common problem



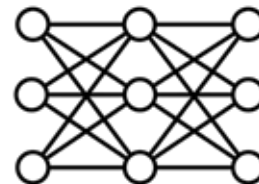
Databases



Machine Images



Container Images



Model Weights

And so much more...

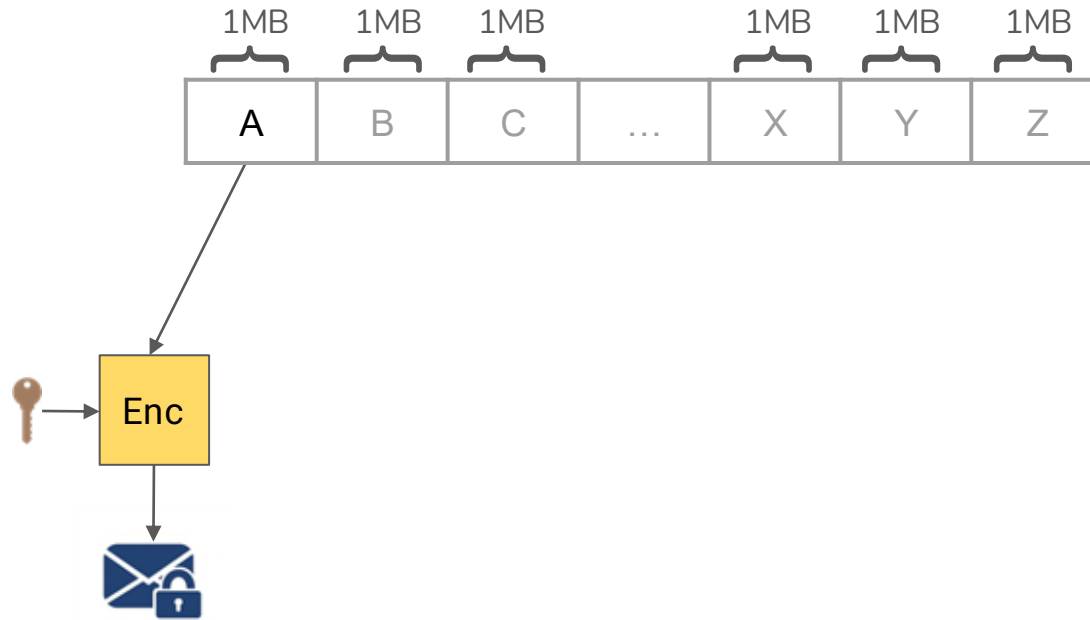
Why not just break it up?

A	B	C	...	X	Y	Z
---	---	---	-----	---	---	---

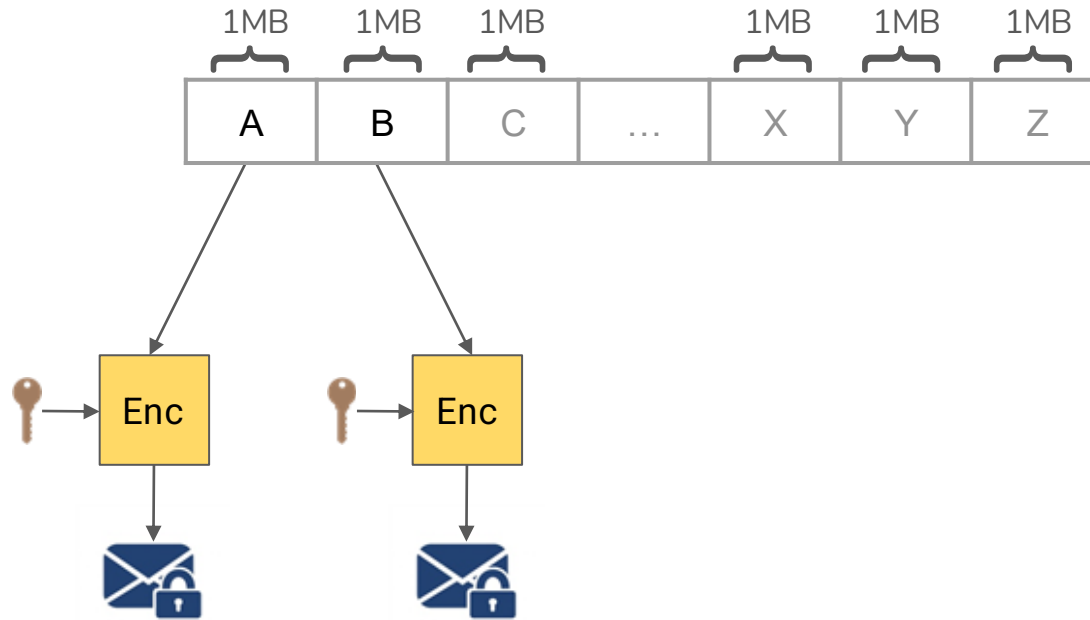
Why not just break it up?



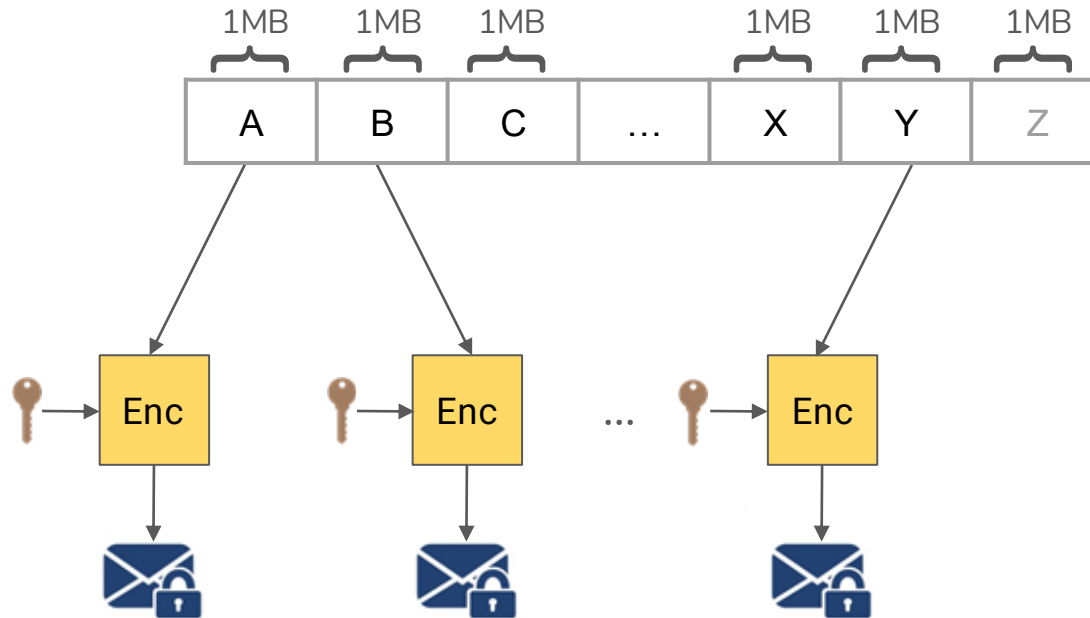
Why not just break it up?



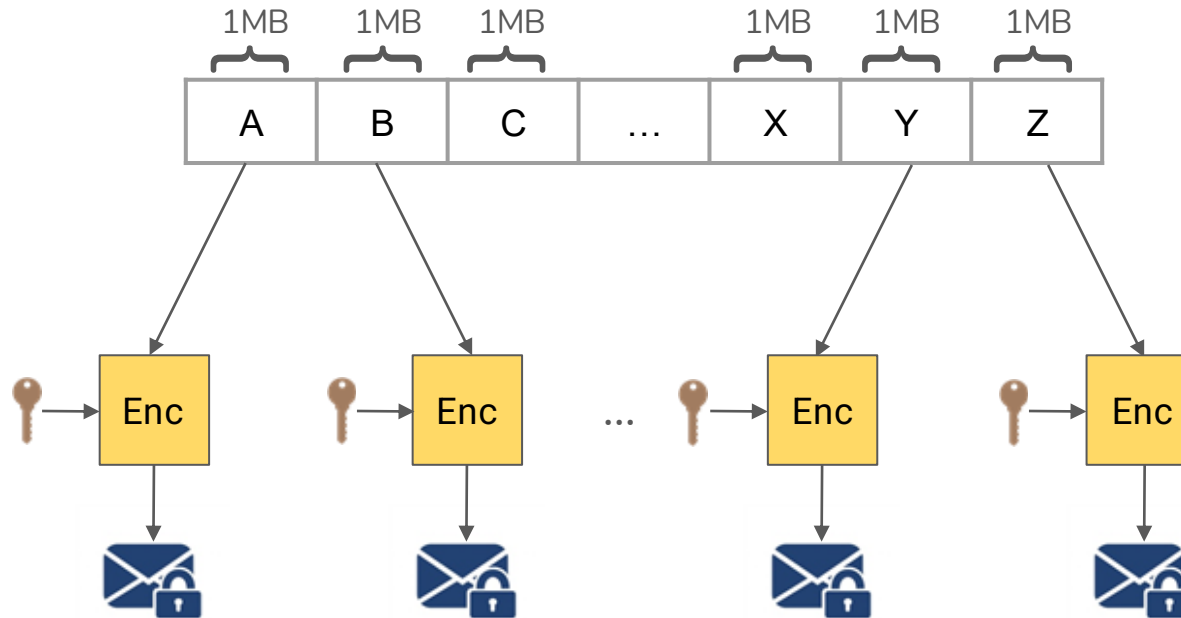
Why not just break it up?



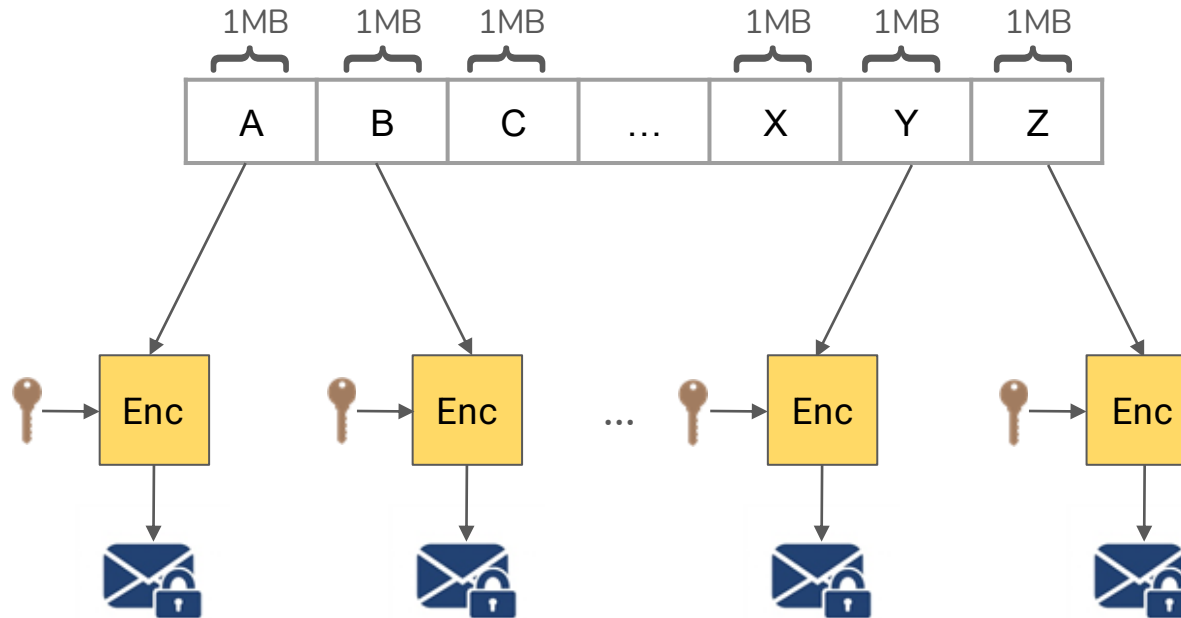
Why not just break it up?



Why not just break it up?



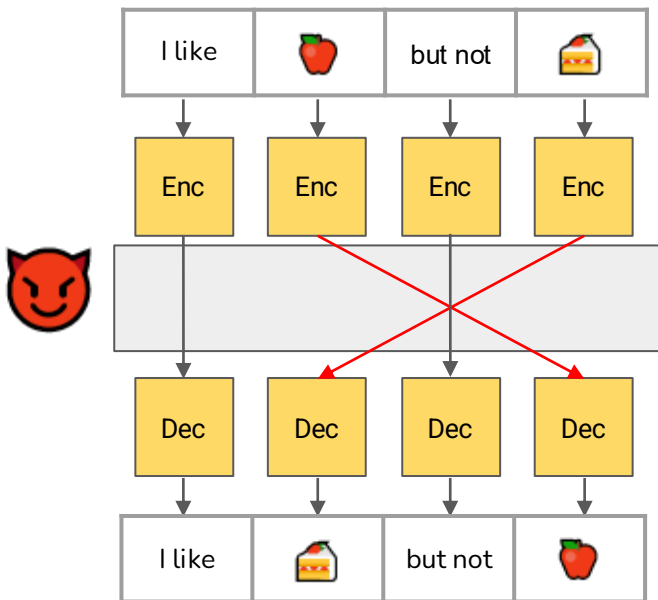
Why not just break it up?



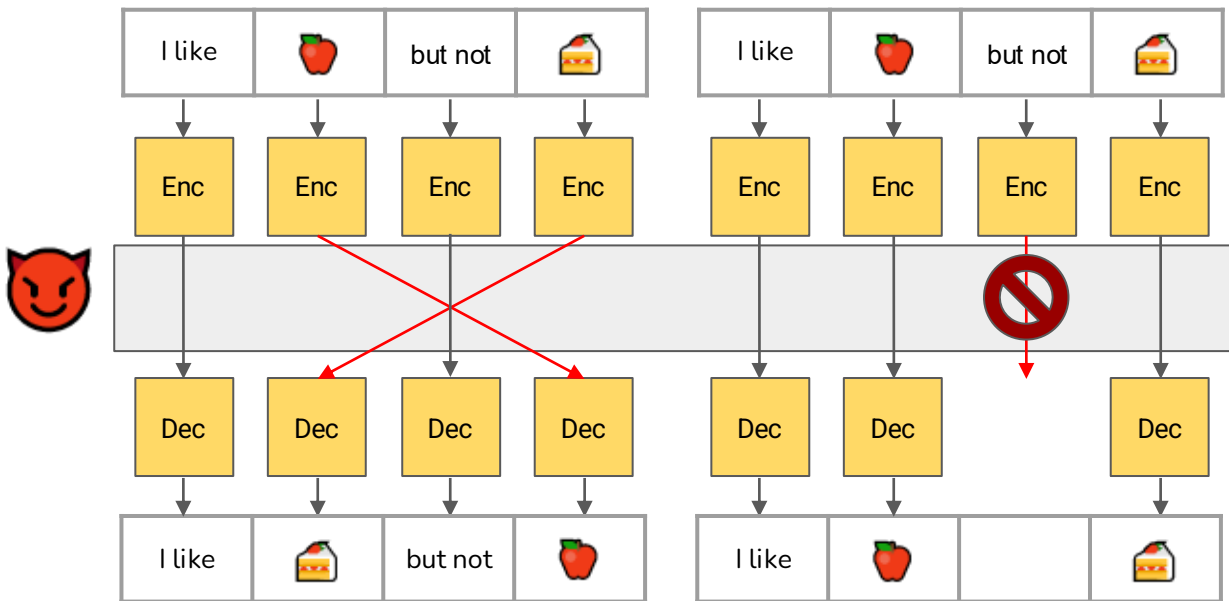
The obvious solution almost works...

Doesn't protect against inter-segment attacks!

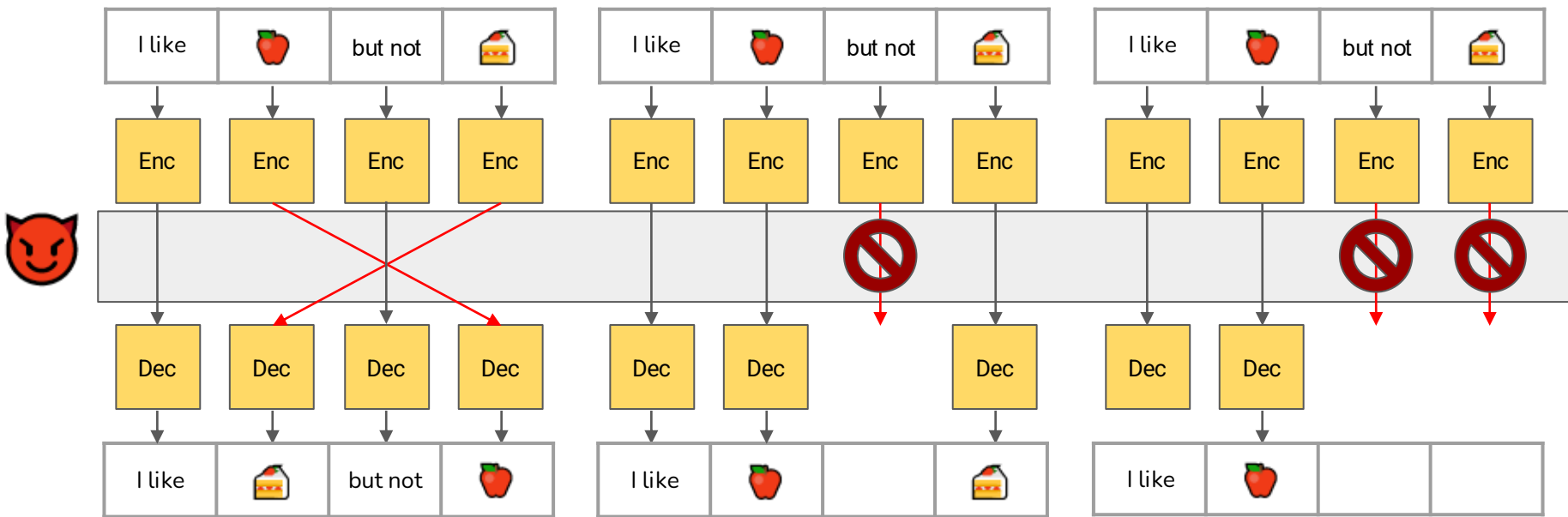
Doesn't protect against inter-segment attacks!



Doesn't protect against inter-segment attacks!



Doesn't protect against inter-segment attacks!



Solved Problem Right?

Literature review time!

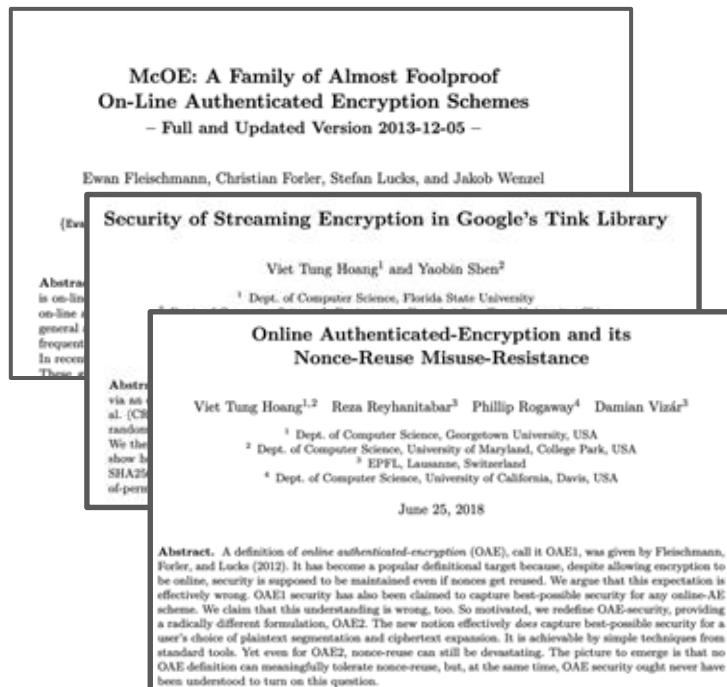
Fleischmann et al. in 2012

Hoang et al. in 2020, 2015

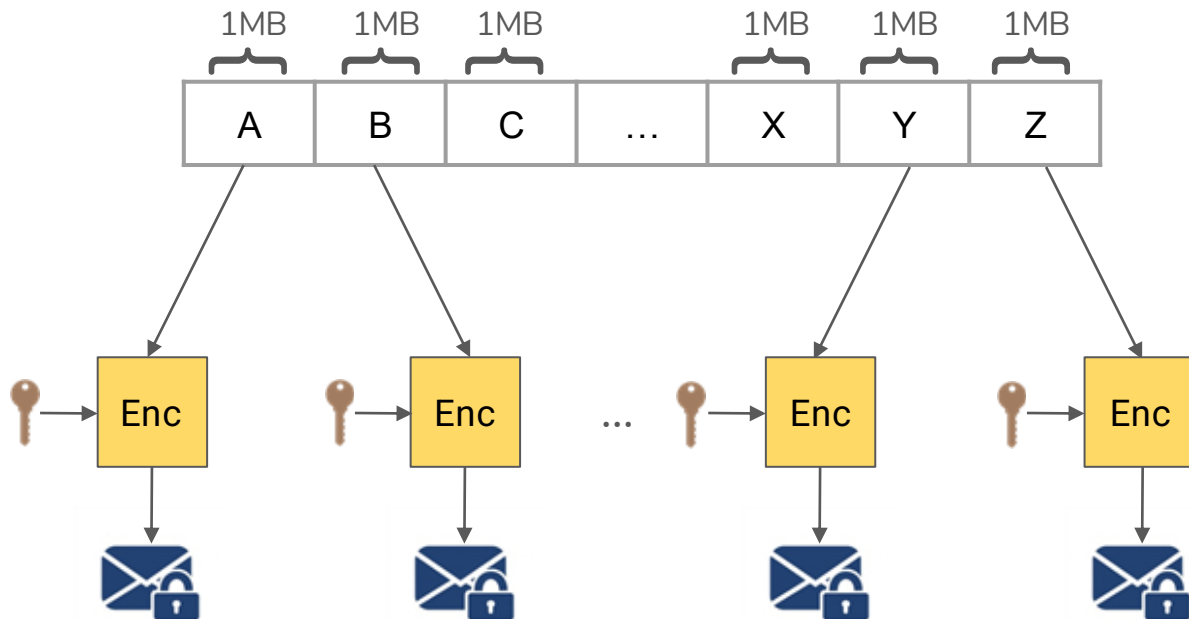
Many encryption libraries over the years.



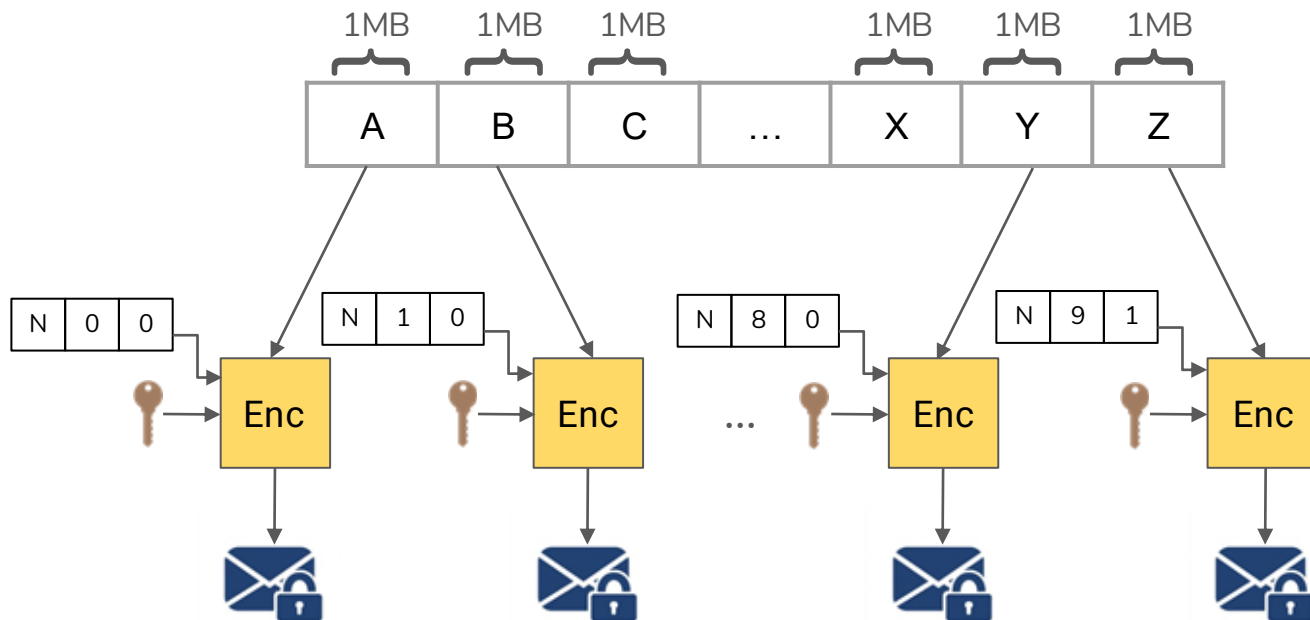
Google Tink



Previous Approaches: STREAM & Tink Streaming

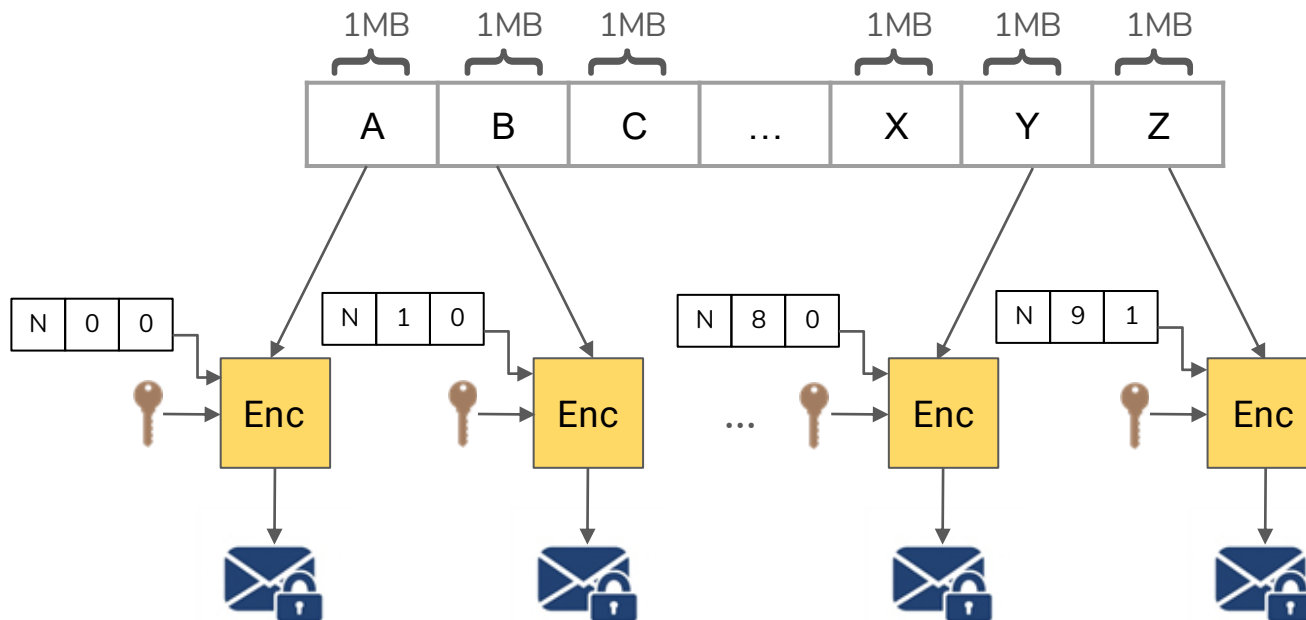


Previous Approaches: STREAM & Tink Streaming



Append **segment numbers** and **terminal bits** to IV

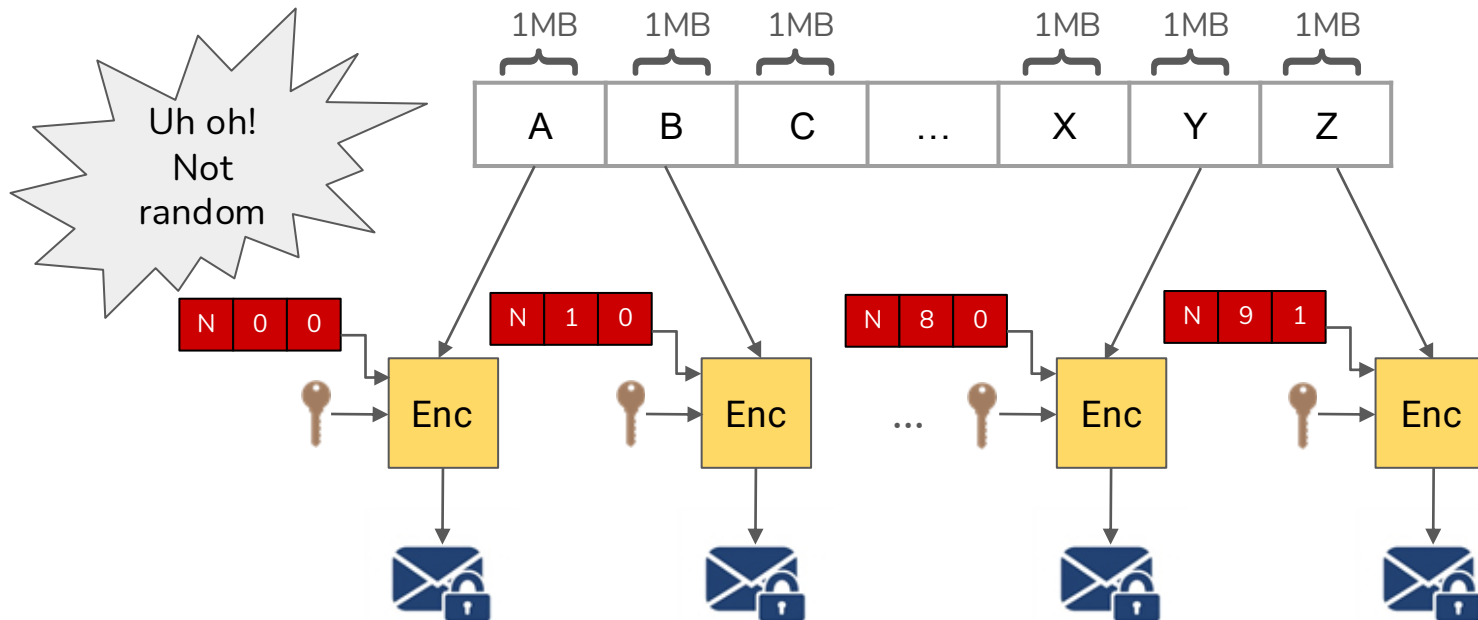
Previous Approaches: STREAM & Tink Streaming



Append **segment numbers** and **terminal bits** to IV

STREAM is secure! [HS20]
Manipulate segments $\Rightarrow$ decrypt with wrong nonce

Previous Approaches: STREAM & Tink Streaming

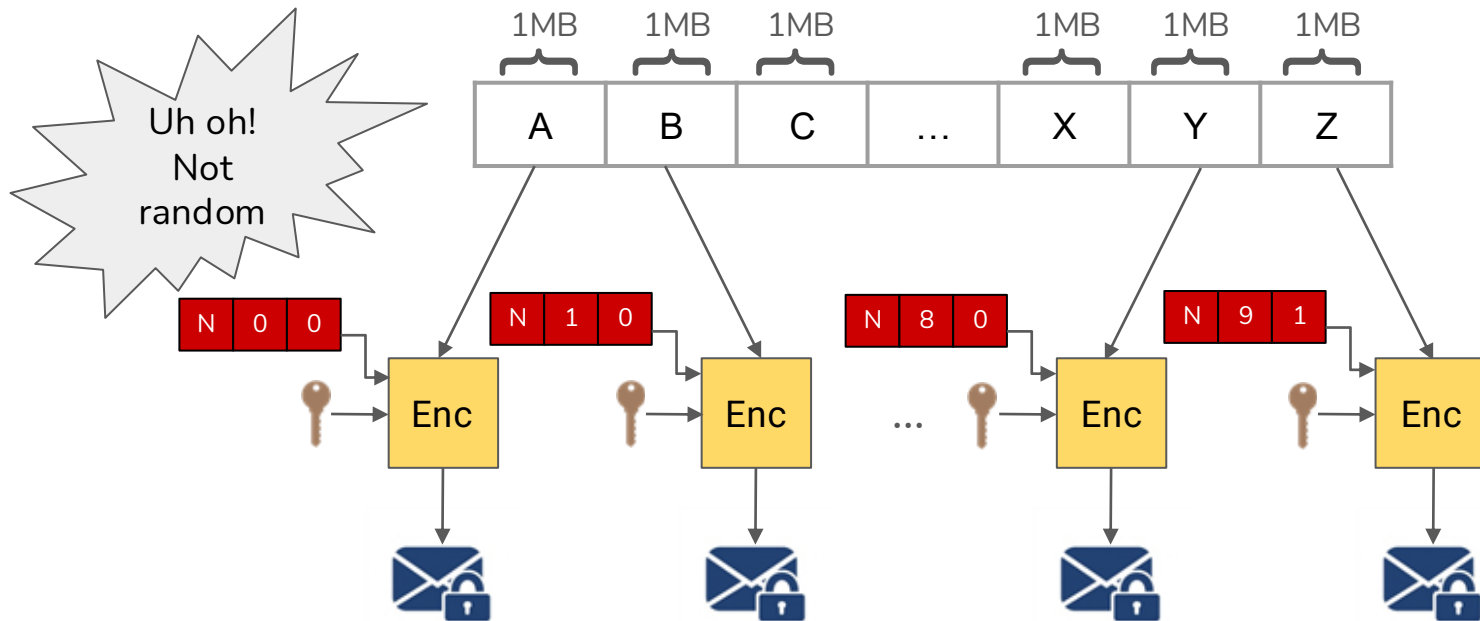


Append **segment numbers** and **terminal bits** to IV

STREAM is secure! [HS20]

Manipulate segments $\Rightarrow$ decrypt with wrong nonce

Previous Approaches: STREAM & Tink Streaming



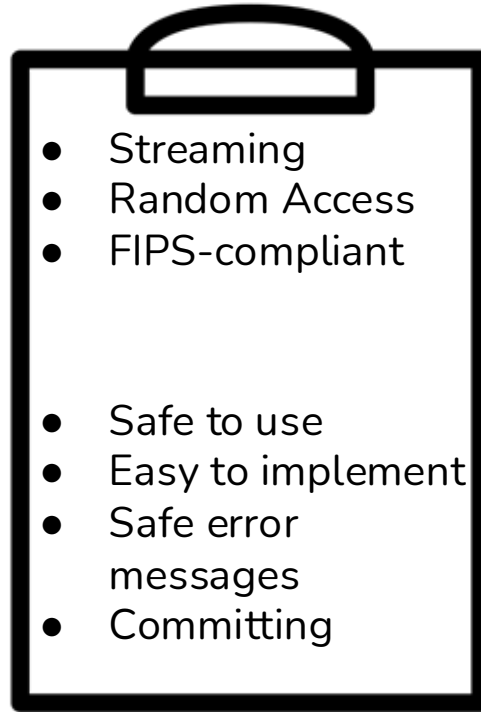
Append **segment numbers** and **terminal bits** to IV

STREAM is secure! [HS20]
Manipulate segments $\Rightarrow$ decrypt with wrong nonce

Random-Access Encrypt is an open question...

We need something new

While we are at it.. let's add some desiderata!



We need something new

While we are at it.. let's add some desiderata!

Safe to use:

Easy to use. Hard to misuse.
Protects against common mistakes.

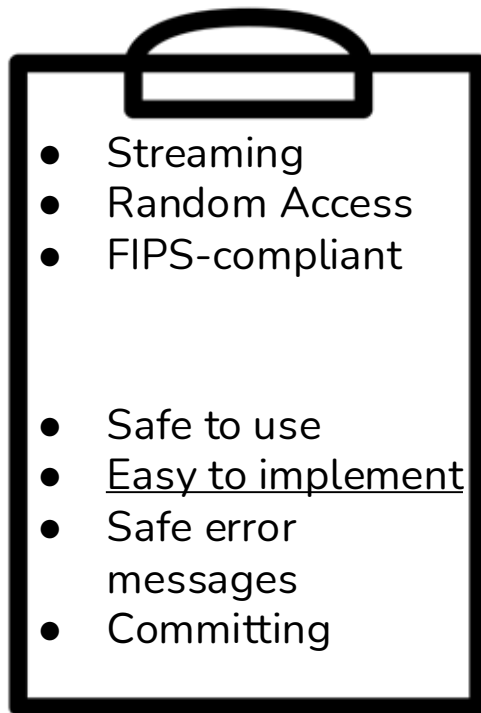


We need something new

While we are at it.. let's add some desiderata!

Safe to use:

Easy to use. Hard to misuse.
Protects against common mistakes.



Easy to implement:

Existing solutions aren't available
in many languages and skilled
crypto-developers are rare.

We need something new

While we are at it.. let's add some desiderata!

Safe to use:

Easy to use. Hard to misuse.
Protects against common mistakes.

Safe error messages:

Developers have trouble
debugging mistakes using
cryptography due to lack of useful
error messages.



Easy to implement:

Existing solutions aren't available
in many languages and skilled
crypto-developers are rare.

We need something new

While we are at it.. let's add some desiderata!

Safe to use:

Easy to use. Hard to misuse.
Protects against common mistakes.

Safe error messages:

Developers have trouble debugging mistakes using cryptography due to lack of useful error messages.



Easy to implement:

Existing solutions aren't available in many languages and skilled crypto-developers are rare.

Committing:

Modern additional security property. Ensures a single ciphertext can only decrypt to a single plaintext.

An aerial photograph of a large, irregularly shaped ice floe floating in dark, choppy water. The ice floe is a light blue/white color, contrasting with the dark water. The text is overlaid on a semi-transparent dark rectangle in the upper half of the image.

FLOE

Fast Lightweight Online Encryption

Photo by David Rubin
Used by permission



FLOE

Fast Lightweight Online Encryption

Floe: (Noun)

- A low, flat mass of floating ice.

~ Wiktionary

Photo by David Rubin
Used by permission

FLOE

Fast Lightweight Online Encryption

Floe: (Noun)

- A low, flat mass of floating ice.

~ Wiktionary

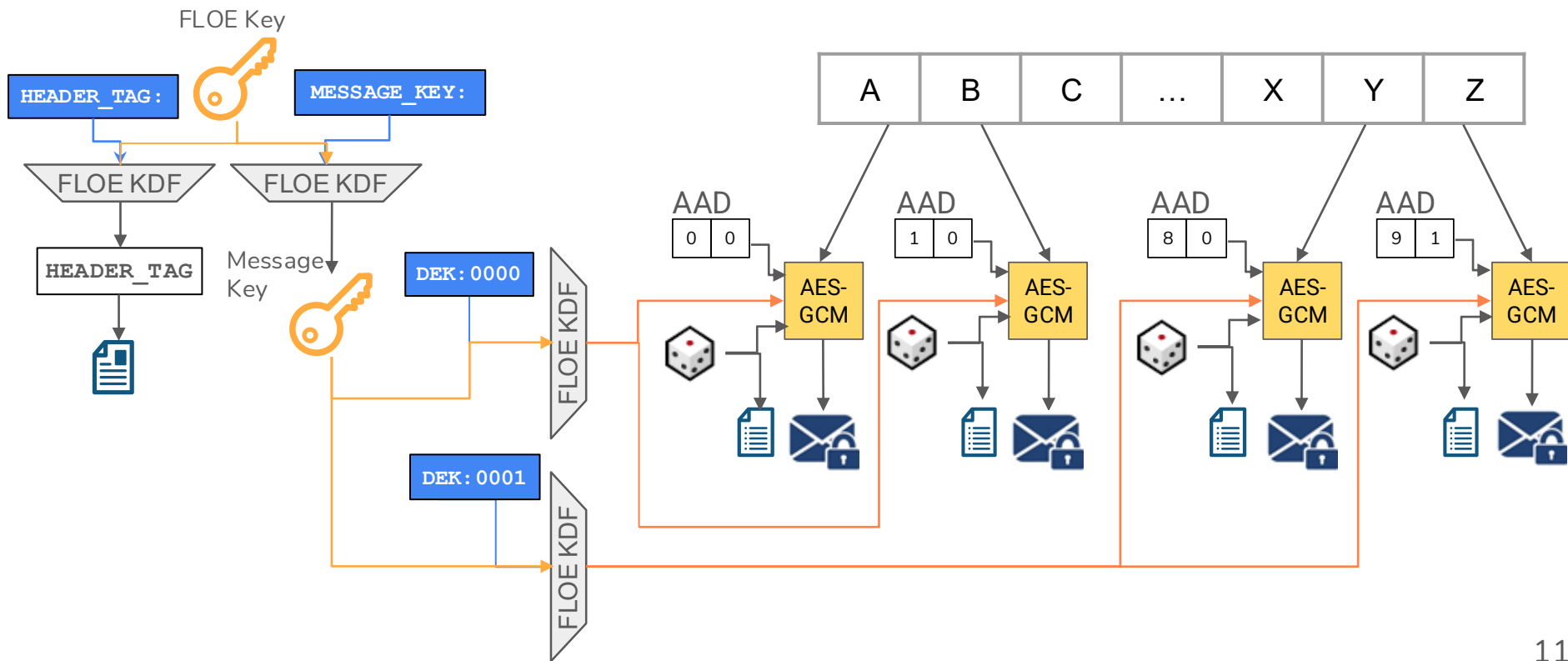
Floe: (Noun)

- A modern efficient encryption algorithm.

~ This paper

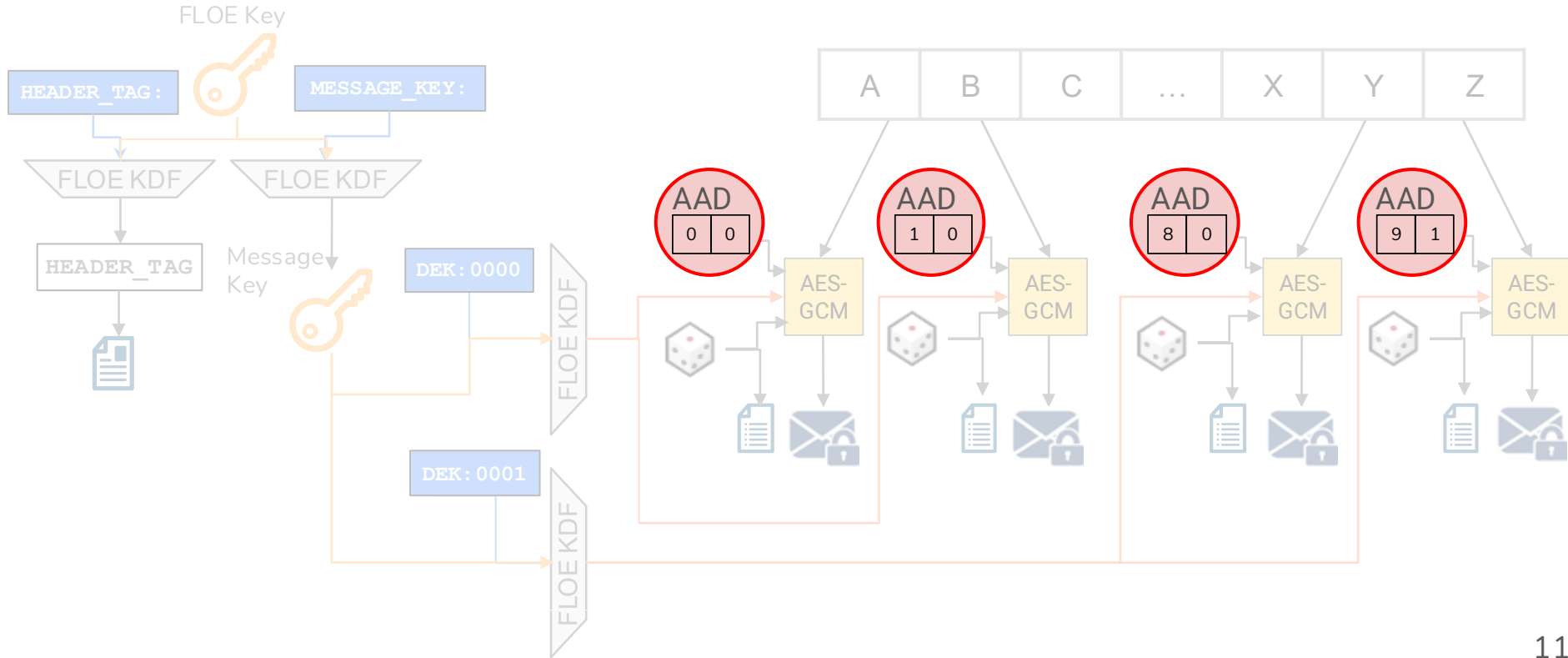
Photo by David Rubin
Used by permission

FLOE



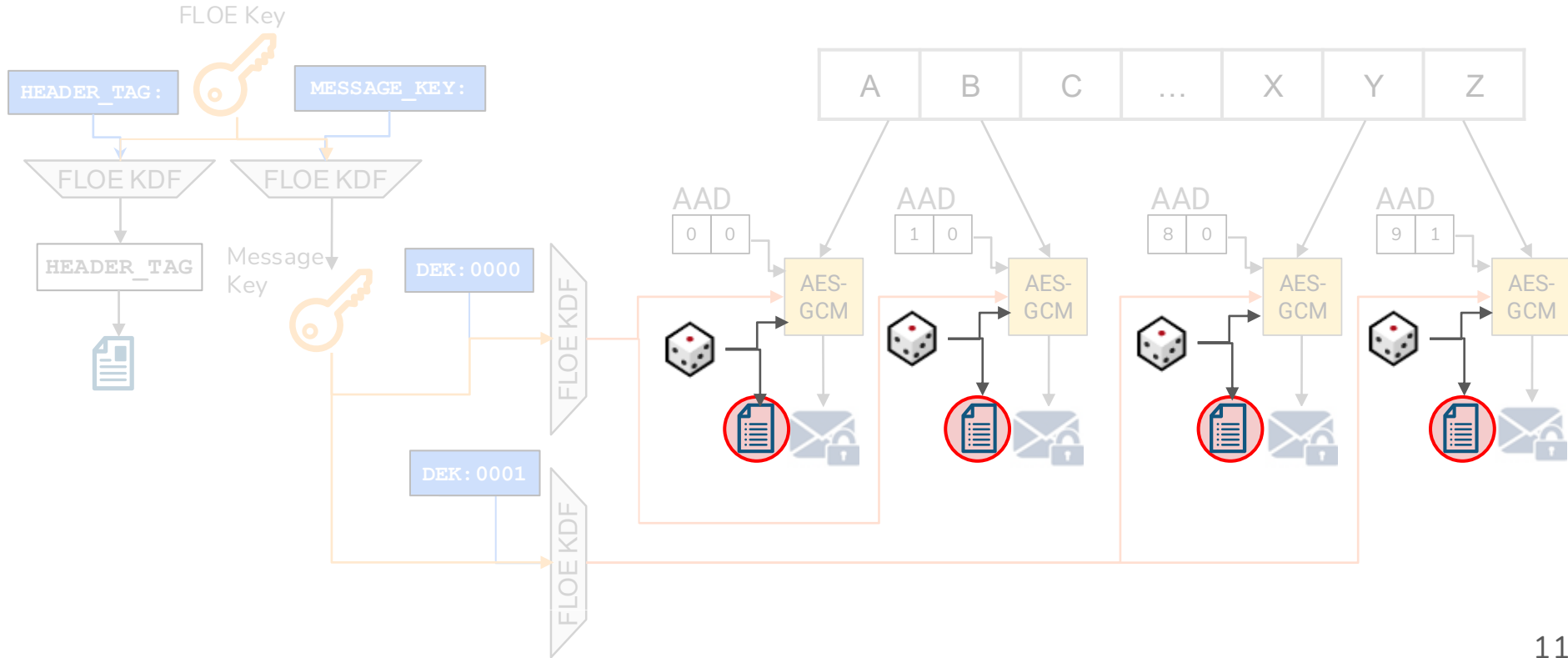
FLOE

Segment location is moved to the AAD



FLOE

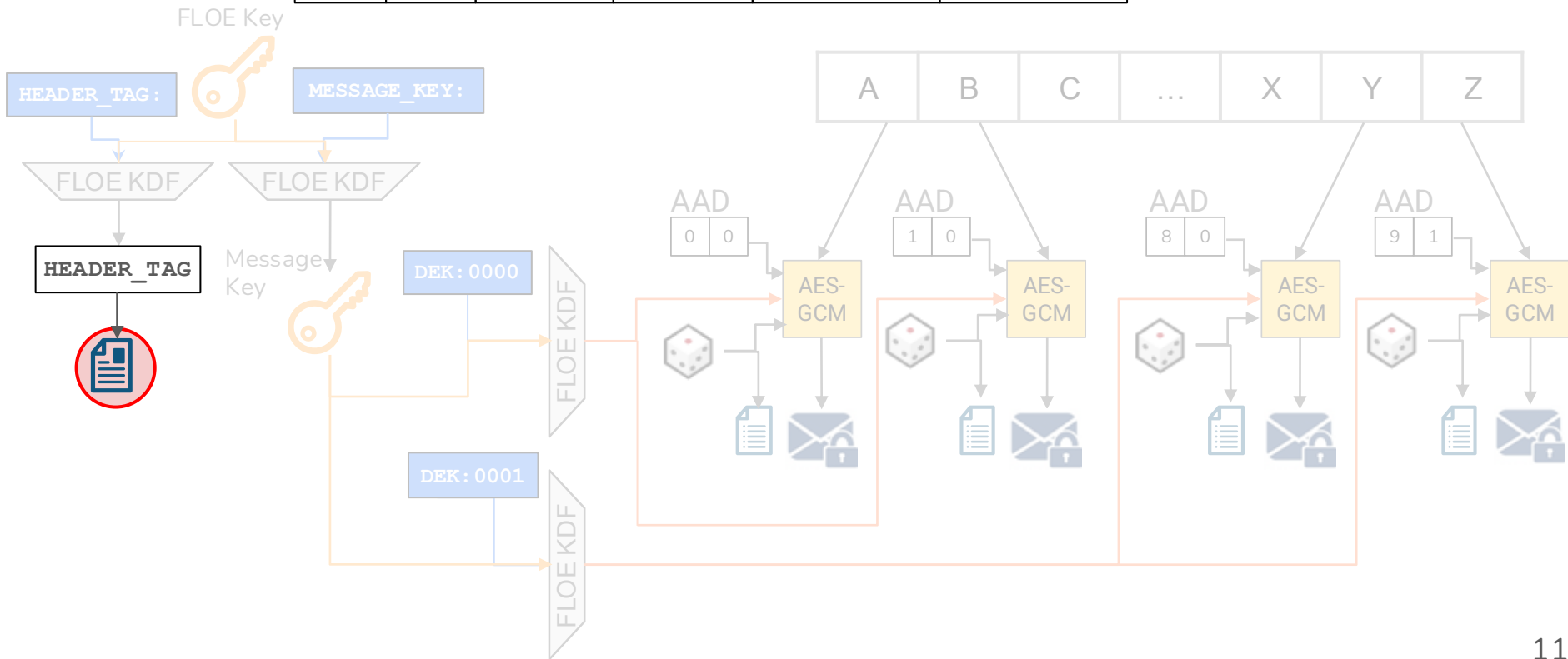
Segment headers to encode random
IVs and simplify streaming decryption



FLOE

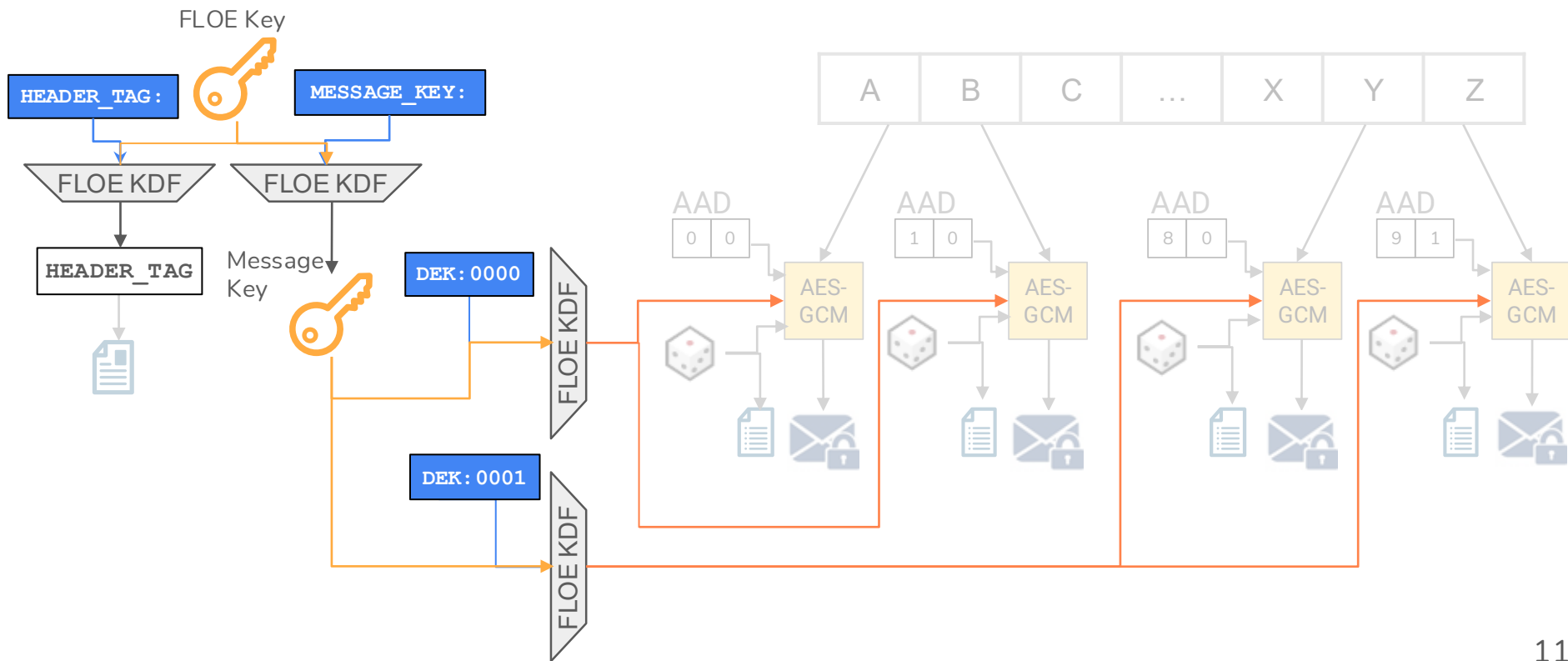
Ciphertext header for commitment, parameters, and main IV

AEAD	KDF	SEG_LEN	IV_LEN	IV	HEADER_TAG
------	-----	---------	--------	----	------------



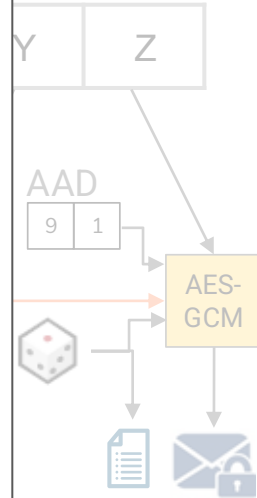
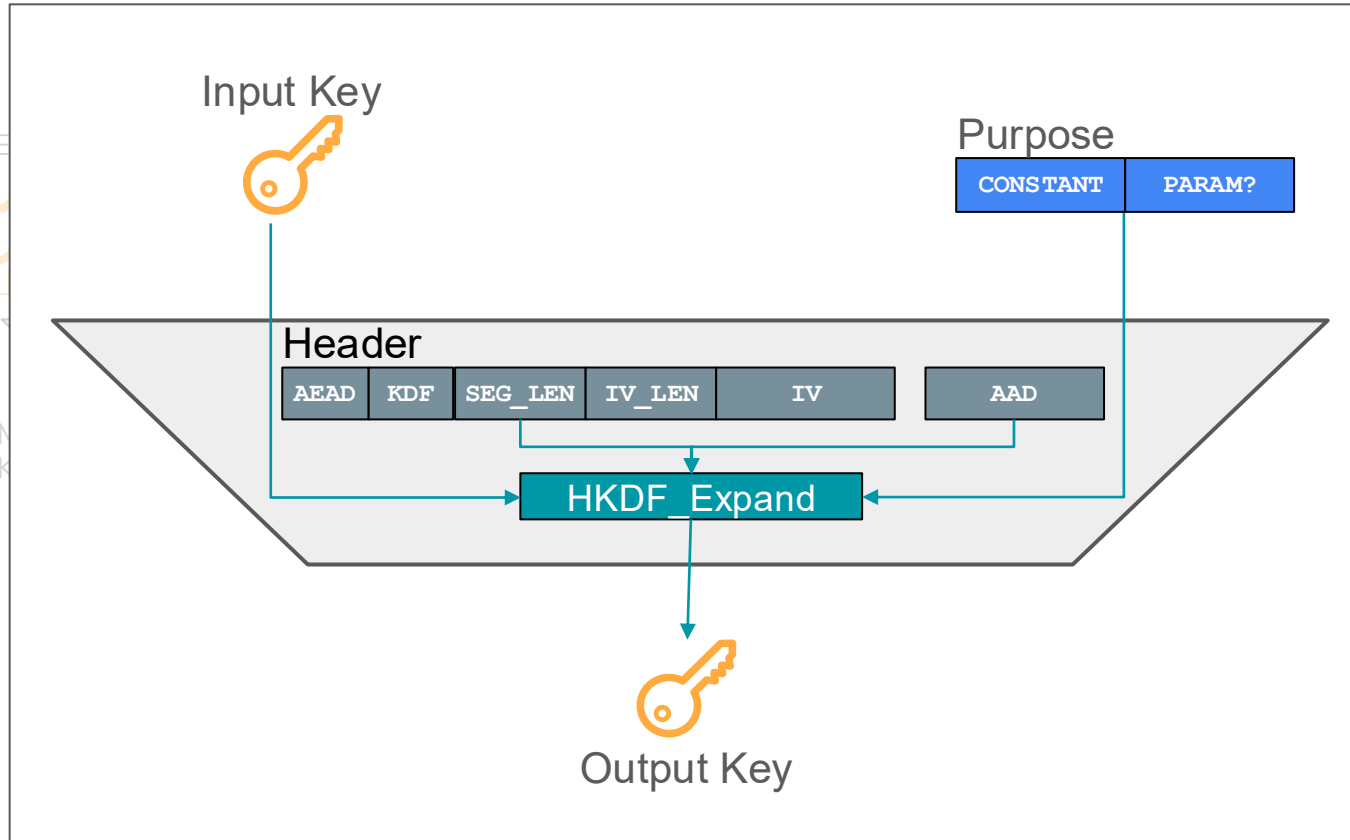
FLOE

Key Schedule ties things together and supports exabytes under a single key

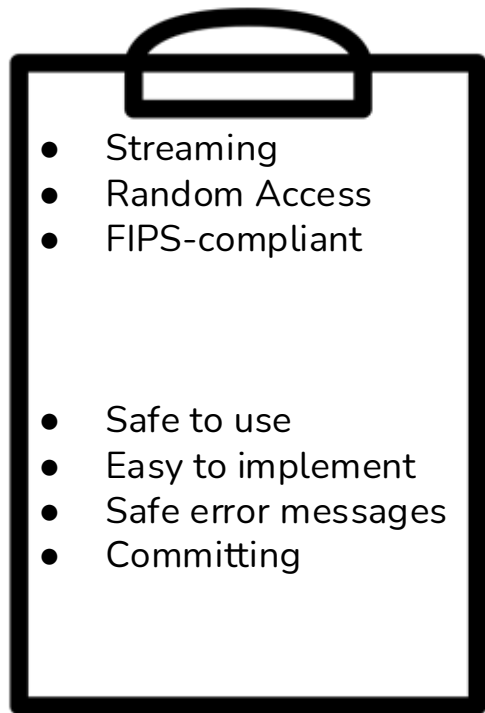


FLOE

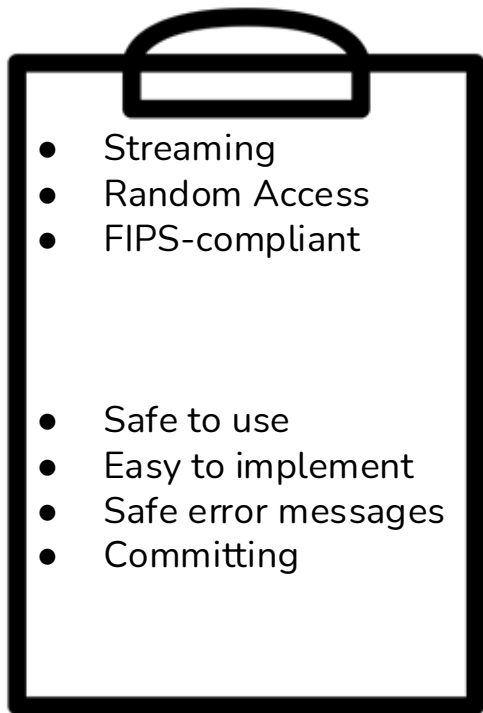
The KDF includes all parameters and main IV



FLOE solves our problems!



FLOE solves our problems!



Is FLOE secure?

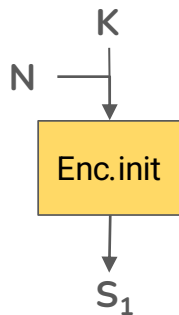
What does security mean in our setting?

What does security mean in our setting?

1. In-order encryption and decryption (Hoang et al., 2015)
 - a. Primitive: online AE scheme
 - b. Security: nOAE

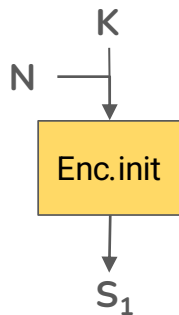
What does security mean in our setting?

1. In-order encryption and decryption (Hoang et al., 2015)
 - a. Primitive: online AE scheme
 - b. Security: nOAE



What does security mean in our setting?

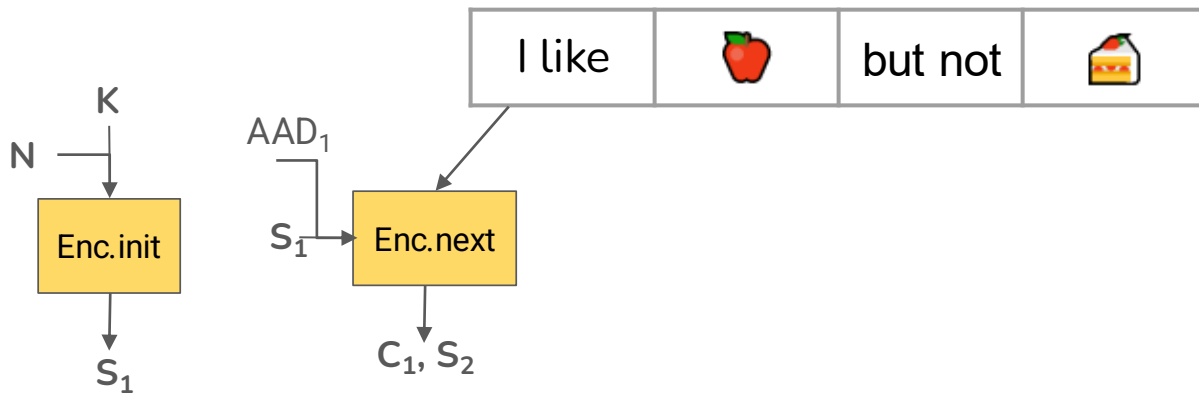
1. In-order encryption and decryption (Hoang et al., 2015)
 - a. Primitive: online AE scheme
 - b. Security: nOAE



I like		but not	
--------	--	---------	---

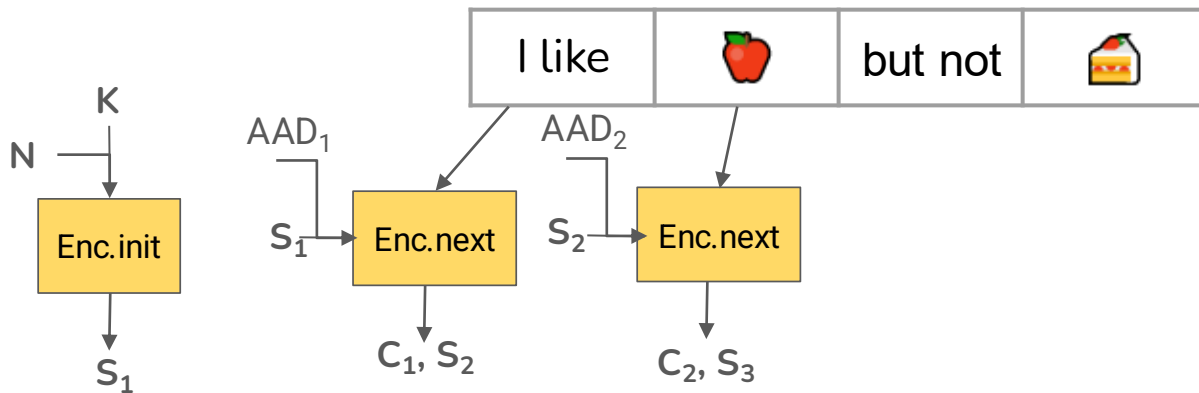
What does security mean in our setting?

1. In-order encryption and decryption (Hoang et al., 2015)
 - a. Primitive: online AE scheme
 - b. Security: nOAE



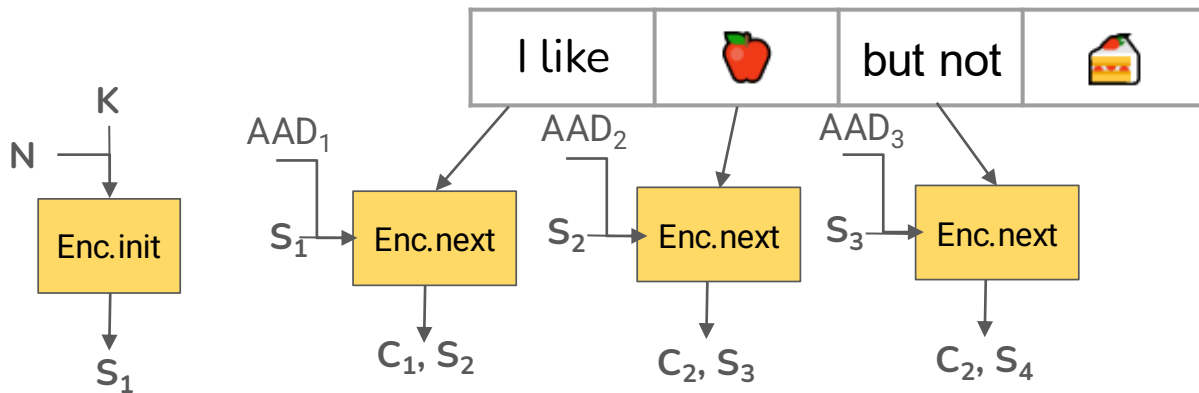
What does security mean in our setting?

1. In-order encryption and decryption (Hoang et al., 2015)
 - a. Primitive: online AE scheme
 - b. Security: nOAE



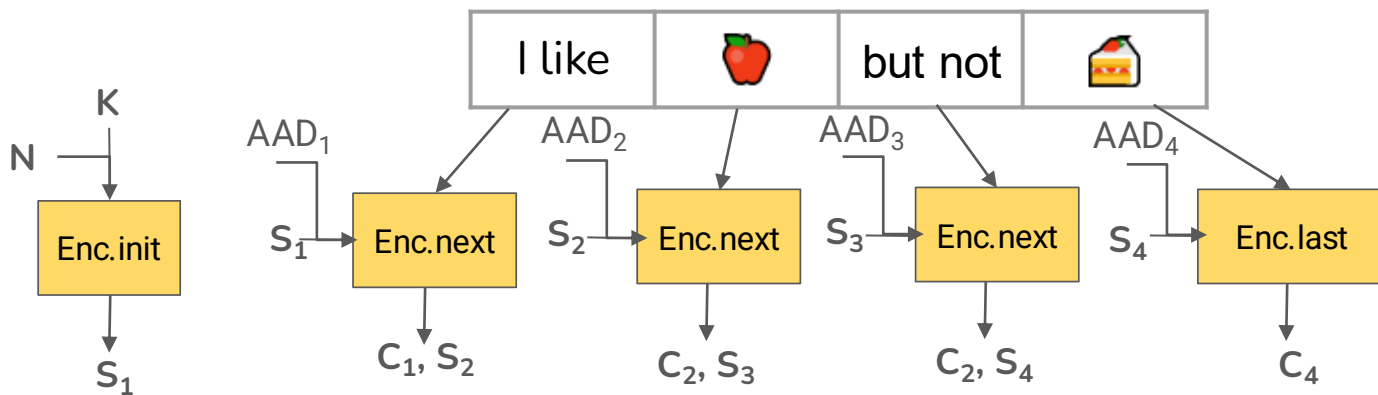
What does security mean in our setting?

1. In-order encryption and decryption (Hoang et al., 2015)
 - a. Primitive: online AE scheme
 - b. Security: nOAE



What does security mean in our setting?

1. In-order encryption and decryption (Hoang et al., 2015)
 - a. Primitive: online AE scheme
 - b. Security: nOAE

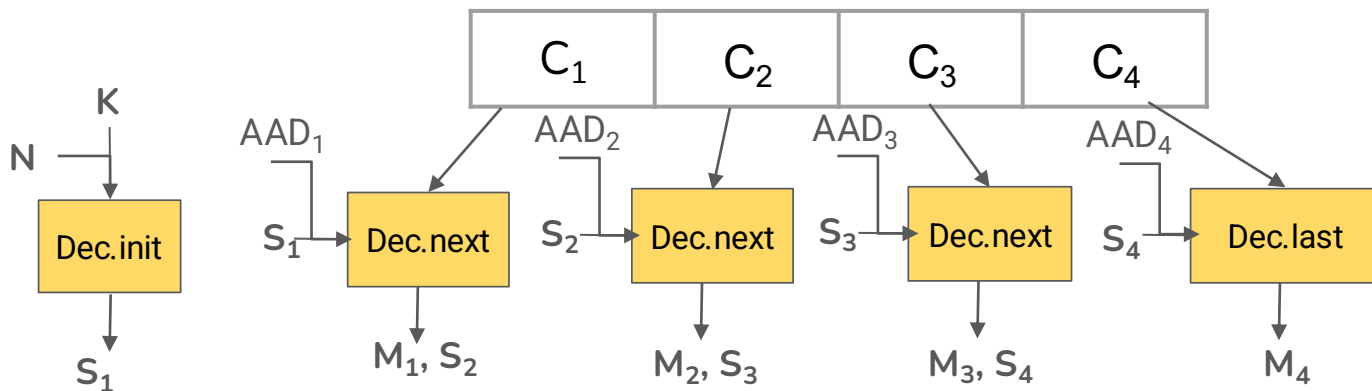


What does security mean in our setting?

1. In-order encryption and decryption (Hoang et al., 2015)
 - a. Primitive: online AE scheme
 - b. Security: nOAE
2. In-order encryption, random-access decryption (Hoang and Shen, 2020)
 - a. Primitive: “canonical” online AE scheme
 - b. Security: nOAE2

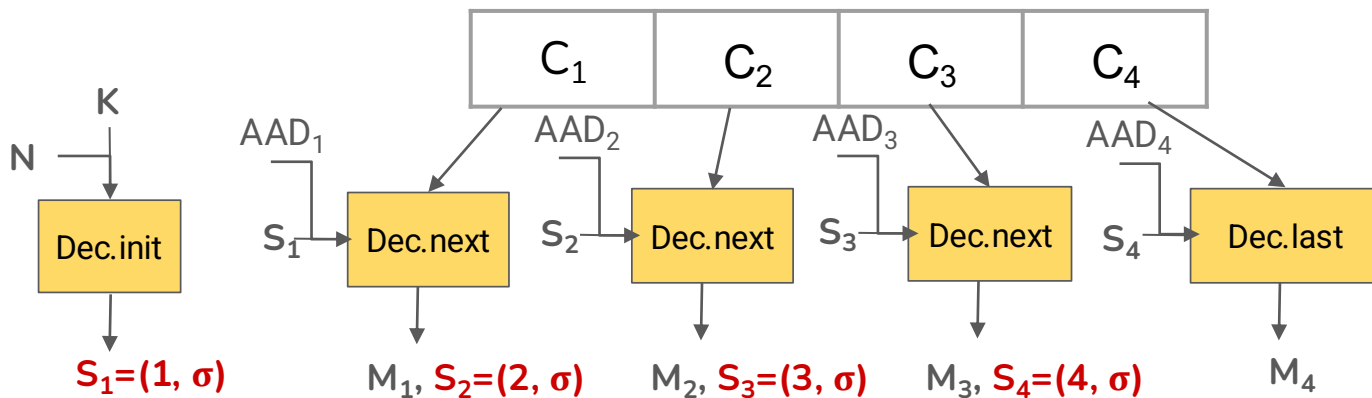
What does security mean in our setting?

1. In-order encryption and decryption (Hoang et al., 2015)
 - a. Primitive: online AE scheme
 - b. Security: nOAE
2. In-order encryption, random-access decryption (Hoang and Shen, 2020)
 - a. Primitive: “canonical” online AE scheme
 - b. Security: nOAE2



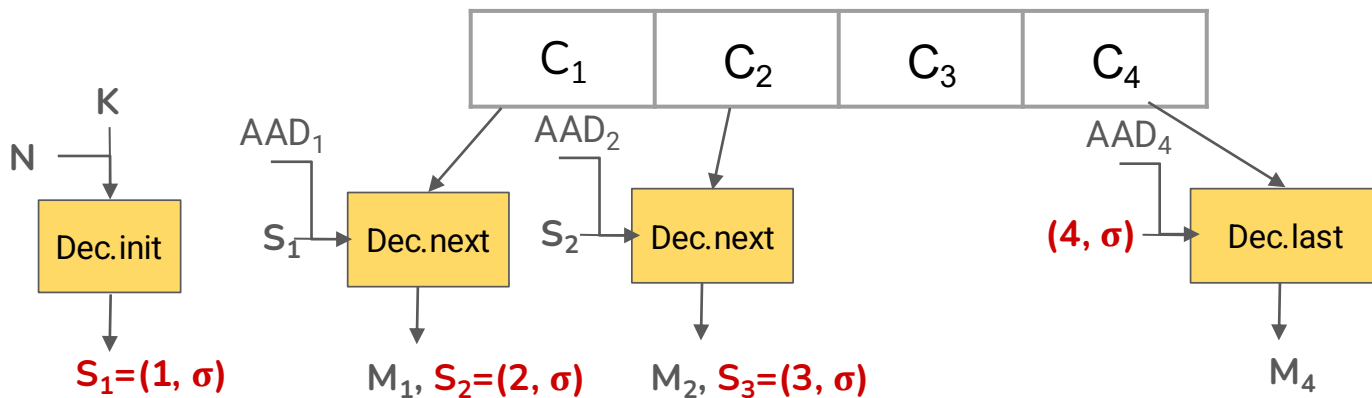
What does security mean in our setting?

1. In-order encryption and decryption (Hoang et al., 2015)
 - a. Primitive: online AE scheme
 - b. Security: nOAE
2. In-order encryption, random-access decryption (Hoang and Shen, 2020)
 - a. Primitive: “canonical” online AE scheme
 - b. Security: nOAE2



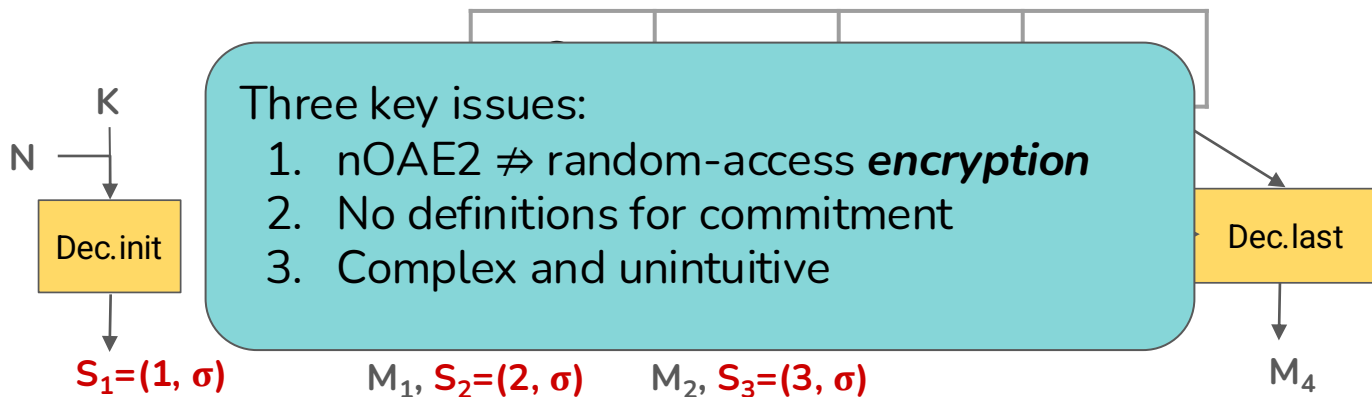
What does security mean in our setting?

1. In-order encryption and decryption (Hoang et al., 2015)
 - a. Primitive: online AE scheme
 - b. Security: nOAE
2. In-order encryption, random-access decryption (Hoang and Shen, 2020)
 - a. Primitive: “canonical” online AE scheme
 - b. Security: nOAE2



What does security mean in our setting?

1. In-order encryption and decryption (Hoang et al., 2015)
 - a. Primitive: online AE scheme
 - b. Security: nOAE
2. In-order encryption, random-access decryption (Hoang and Shen, 2020)
 - a. Primitive: “canonical” online AE scheme
 - b. Security: nOAE2

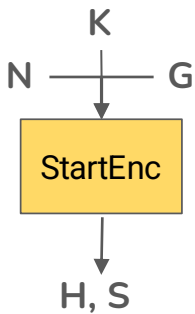


New primitive: random-access authenticated encryption (raAE)

Single segment encryption/decryption process; encode positions explicitly

New primitive: random-access authenticated encryption (raAE)

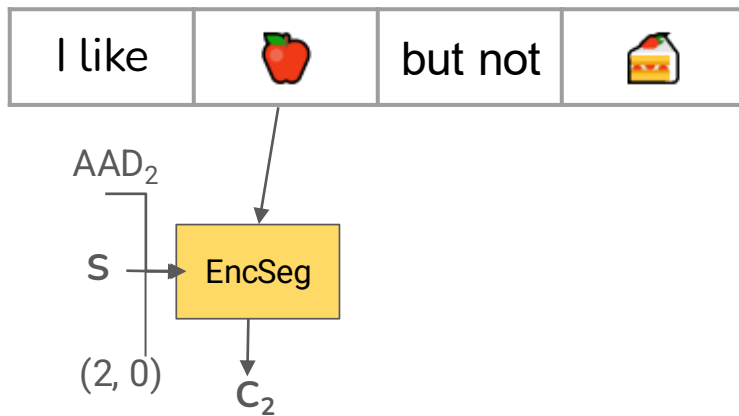
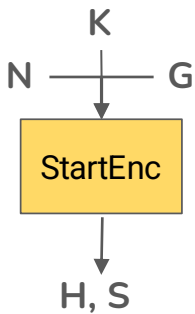
Single segment encryption/decryption process; encode positions explicitly



I like		but not	
--------	--	---------	---

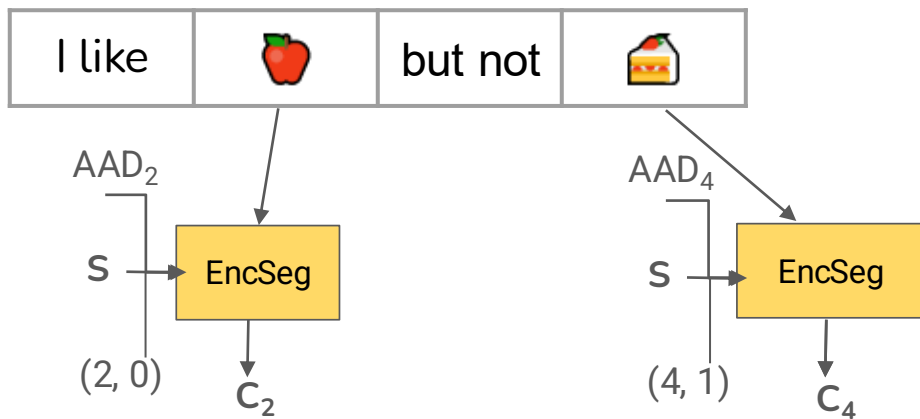
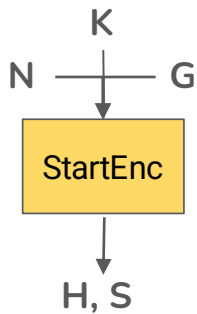
New primitive: random-access authenticated encryption (raAE)

Single segment encryption/decryption process; encode positions explicitly



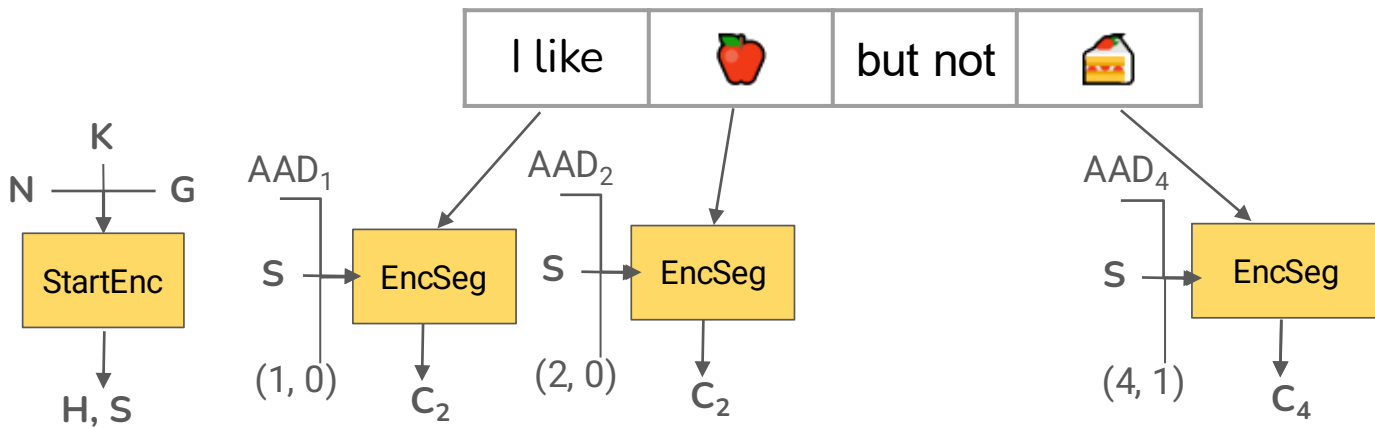
New primitive: random-access authenticated encryption (raAE)

Single segment encryption/decryption process; encode positions explicitly



New primitive: random-access authenticated encryption (raAE)

Single segment encryption/decryption process; encode positions explicitly

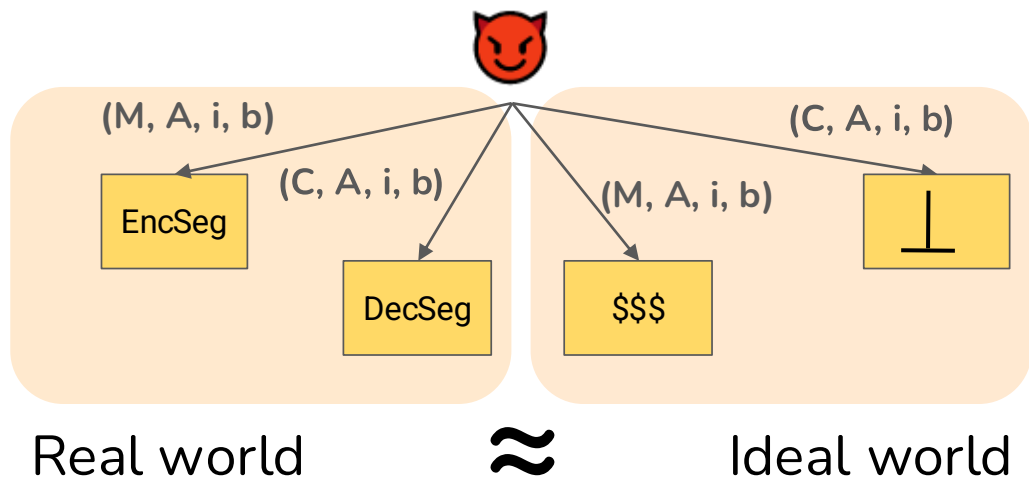


New security notion: random-access real-or-random (ra-ROR)

Standard (multi-user) AEAD security, on [per-segment](#) basis

New security notion: random-access real-or-random (ra-ROR)

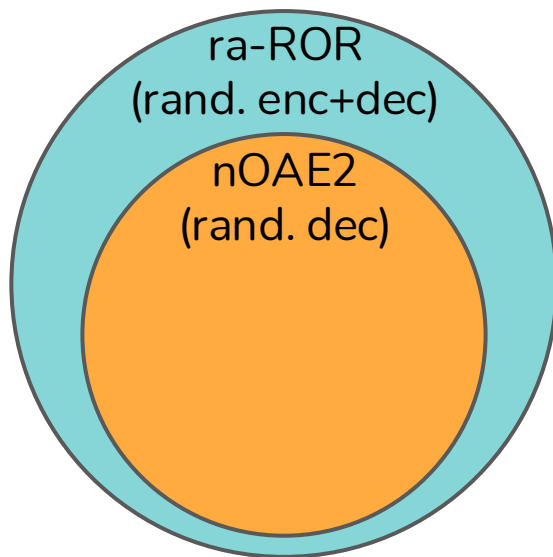
Standard (multi-user) AEAD security, on **per-segment** basis



ra-ROR is strictly stronger than nOAE2

$\text{ra-ROR} \Rightarrow \text{nOAE2}$

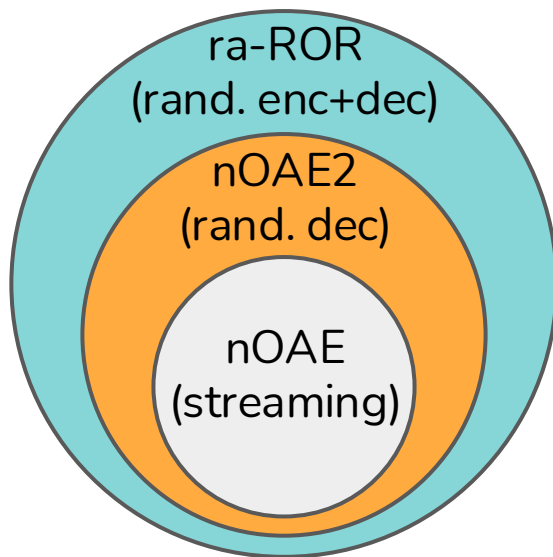
$\text{ra-ROR} \not\Leftarrow \text{nOAE2}$



ra-ROR is strictly stronger than nOAE2

$\text{ra-ROR} \Rightarrow \text{nOAE2}$

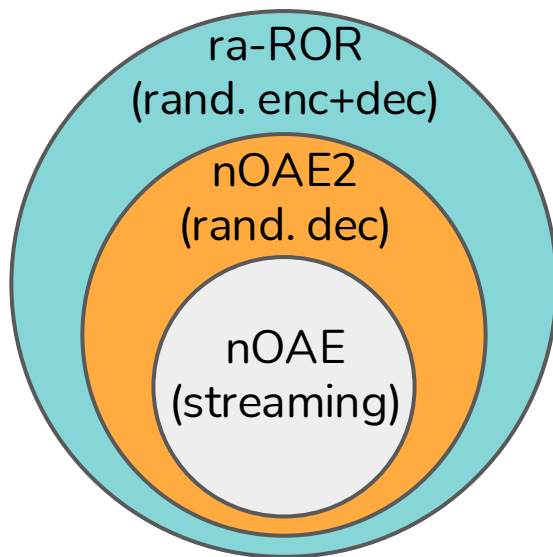
$\text{ra-ROR} \not\Leftarrow \text{nOAE2}$



ra-ROR is strictly stronger than nOAE2

ra-ROR $\Rightarrow$ nOAE2

ra-ROR $\nLeftarrow$ nOAE2



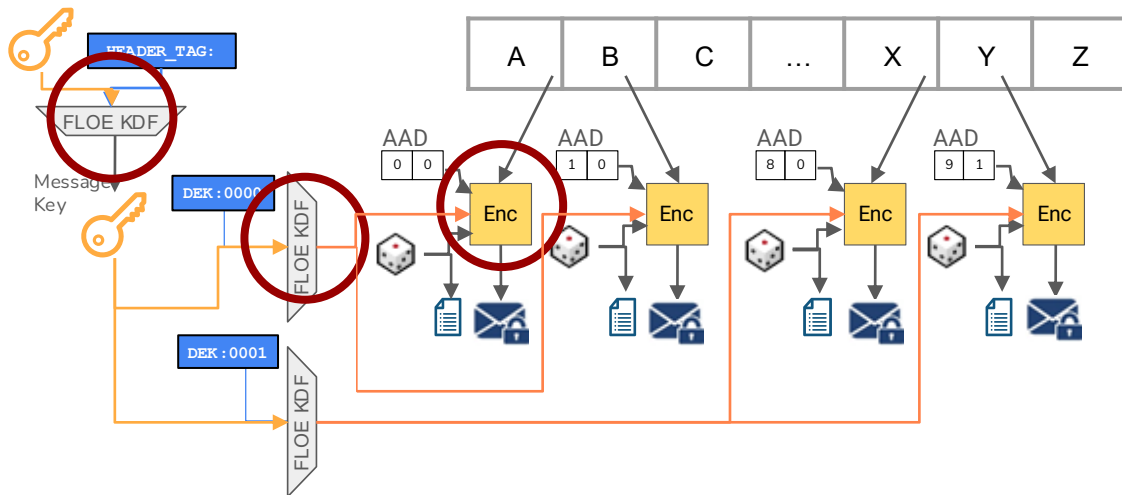
Existing schemes (e.g., STREAM, Tink streaming) **remain secure** under ra-ROR!
 $\Rightarrow$ For example, safe to encrypt segment in parallel

FLOE satisfies ra-ROR security!

ra-ROR = μ -PRF security of F + μ -ROR\$ security of AEAD

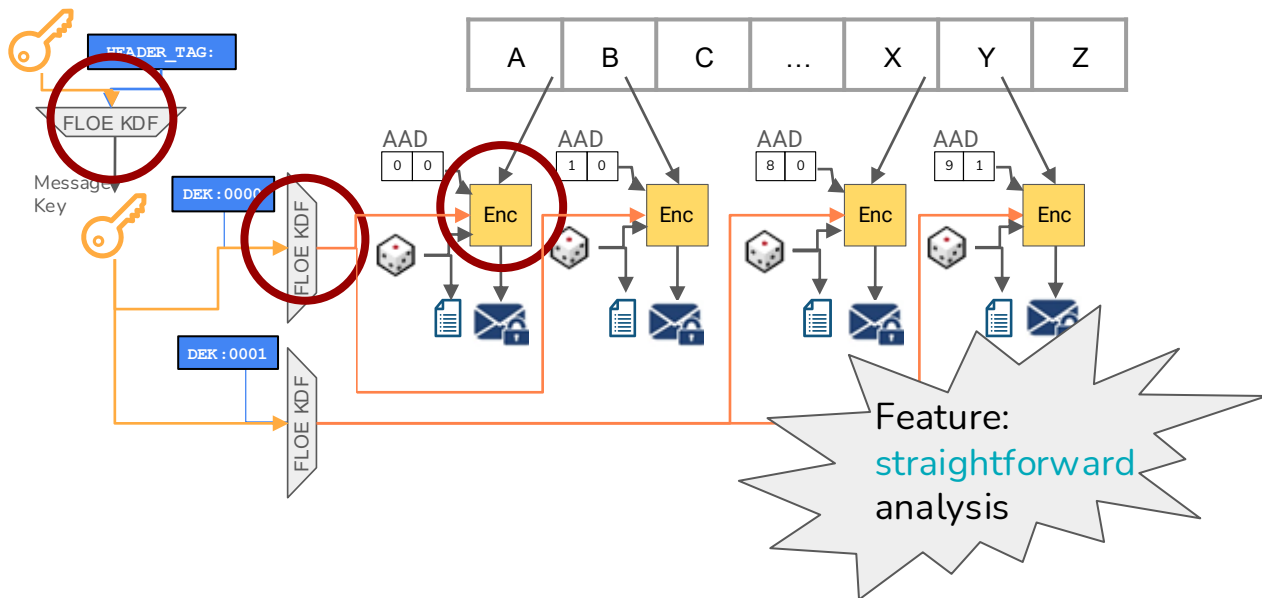
FLOE satisfies ra-ROR security!

ra-ROR = **mu-PRF** security of F + **mu-ROR\$** security of AEAD



FLOE satisfies ra-ROR security!

ra-ROR = **mu-PRF** security of F + **mu-ROR\$** security of AEAD



FLOE security scaling

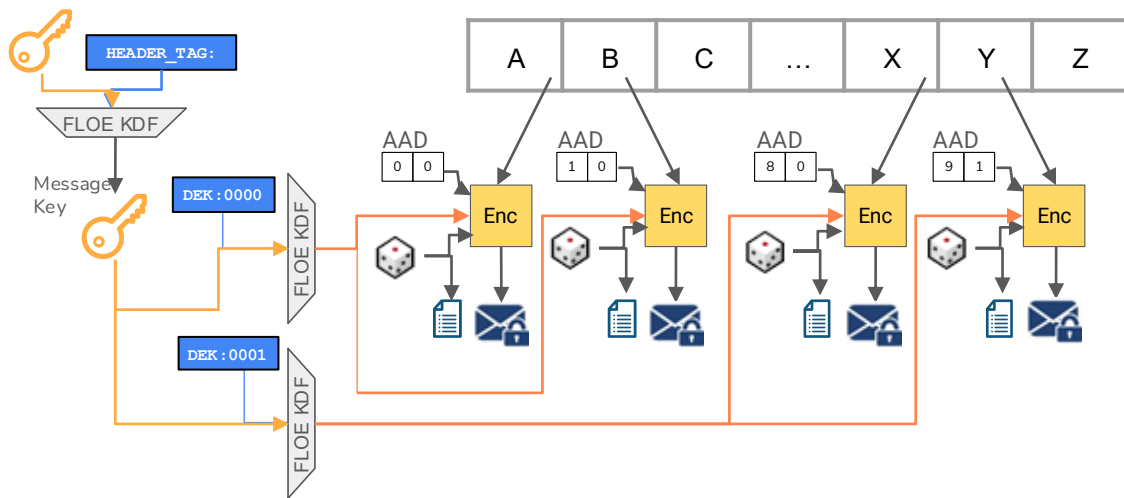
- AES-GCM requires frequent key rotations! Random IVs: 2^{32} messages/key [KCCP24]
 - Lots of prior work on extending key lifespans (DNDK [G24, GR25], Pencil [BD25], XAES-256-GCM [V24], ...)

FLOE security scaling

- AES-GCM requires frequent key rotations! Random IVs: 2^{32} messages/key [KCCP24]
 - Lots of prior work on extending key lifespans (DNDK [G24, GR25], Pencil [BD25], XAES-256-GCM [V24], ...)
- Security (and performance!) of FLOE depends on parameter choices, in particular:
 - Epoch duration and segment size

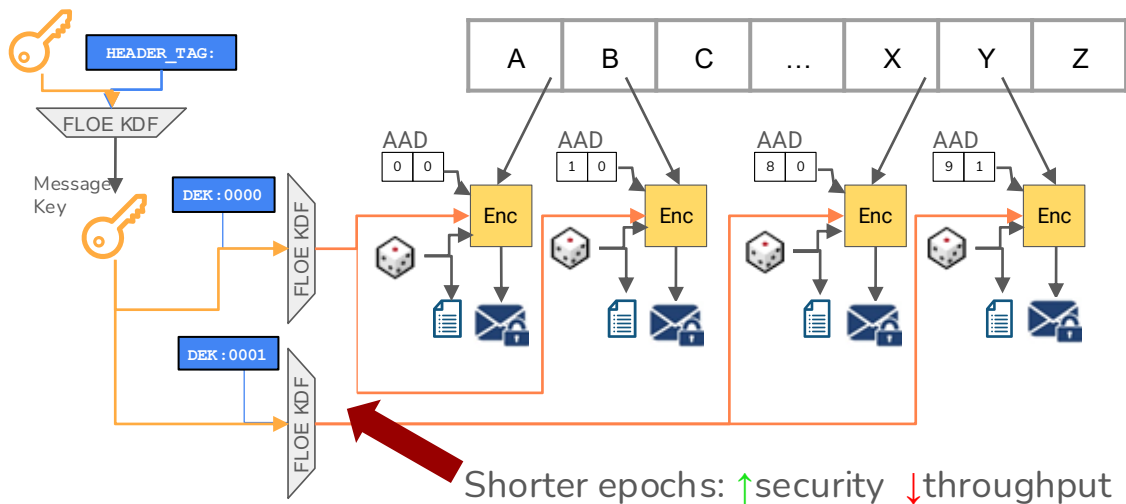
FLOE security scaling

- AES-GCM requires **frequent key rotations**! Random IVs: 2^{32} messages/key [KCCP24]
 - Lots of prior work on **extending** key lifespans (DNDK [G24, GR25], Pencil [BD25], XAES-256-GCM [V24], ...)
- Security (and performance!) of FLOE depends on parameter choices, in particular:
 - **Epoch duration** and **segment size**



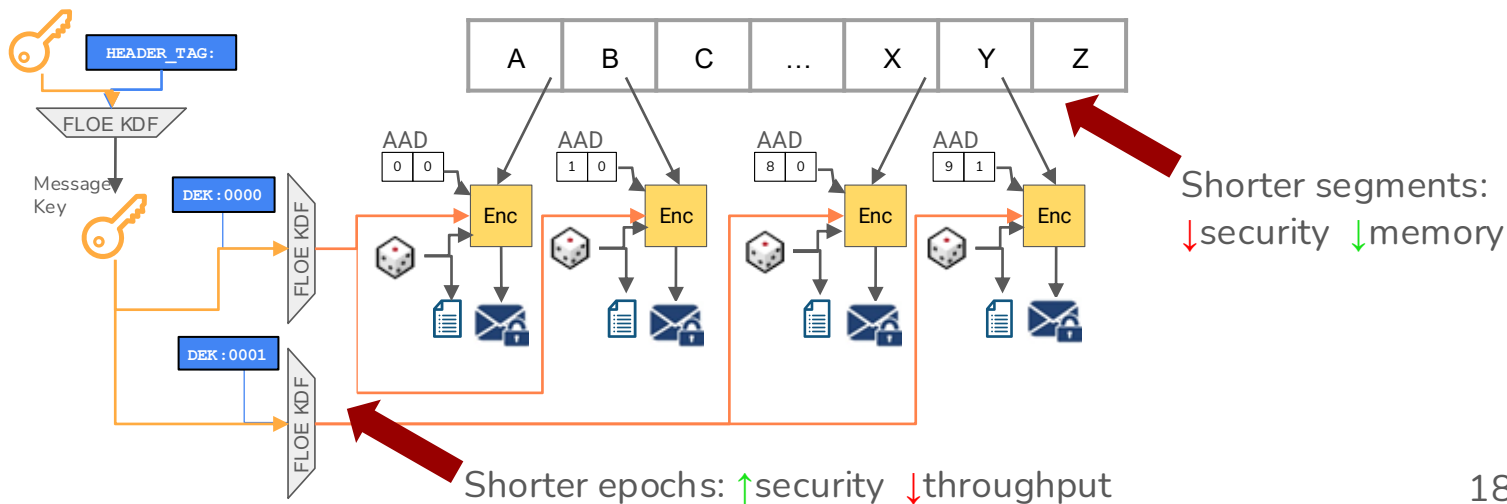
FLOE security scaling

- AES-GCM requires **frequent key rotations**! Random IVs: 2^{32} messages/key [KCCP24]
 - Lots of prior work on **extending** key lifespans (DNDK [G24, GR25], Pencil [BD25], XAES-256-GCM [V24], ...)
- Security (and performance!) of FLOE depends on parameter choices, in particular:
 - **Epoch duration** and **segment size**



FLOE security scaling

- AES-GCM requires **frequent key rotations**! Random IVs: 2^{32} messages/key [KCCP24]
 - Lots of prior work on **extending** key lifespans (DNDK [G24, GR25], Pencil [BD25], XAES-256-GCM [V24], ...)
- Security (and performance!) of FLOE depends on parameter choices, in particular:
 - **Epoch duration** and **segment size**



Commitment for random-access AE

Commitment for random-access AE

- Context commitment (CMT) for AEADs:

Ciphertexts should only be decryptable by one (K, N, A)

Many real-world attacks! [DGRW19, LGR21, ADG⁺22, ...]



Commitment for random-access AE

Many **real-world attacks!** [DGRW19, LGR21, ADG⁺22, ...]

- **Context commitment (CMT)** for AEADs:
Ciphertexts should only be decryptable by **one** (K, N, A)
- New security notion: **random-access context commitment (ra-CMT)**
 - Definitional subtlety: **position** vs **positionless** commitment

Commitment for random-access AE

Many **real-world attacks!** [DGRW19, LGR21, ADG⁺22, ...]

- **Context commitment (CMT)** for AEADs:
Ciphertexts should only be decryptable by **one** (K, N, A)
- New security notion: **random-access context commitment (ra-CMT)**
 - Definitional subtlety: **position** vs **positionless** commitment

Scheme	Positionless ra-CMT	Position ra-CMT
STREAM	CMT of AEAD	CMT of AEAD
Tink streaming	CMT of AEAD	CMT of AEAD
FLOE	Coll. resistance of KDF	CMT of AEAD

Commitment for random-access AE

Many **real-world attacks!** [DGRW19, LGR21, ADG⁺22, ...]

- **Context commitment (CMT)** for AEADs:
Ciphertexts should only be decryptable by **one** (K, N, A)
- New security notion: **random-access context commitment (ra-CMT)**
 - Definitional subtlety: **position** vs **positionless** commitment

Scheme	Positionless ra-CMT	Position ra-CMT
STREAM	CMT of AEAD	CMT of AEAD
Tink streaming	CMT of AEAD	CMT of AEAD
FLOE	Coll. resistance of KDF	CMT of AEAD

⇒ ra-CMT even using
AES-GCM!

FLOE Performance

FLOE versus AES-GCM

“Streaming” Encryption

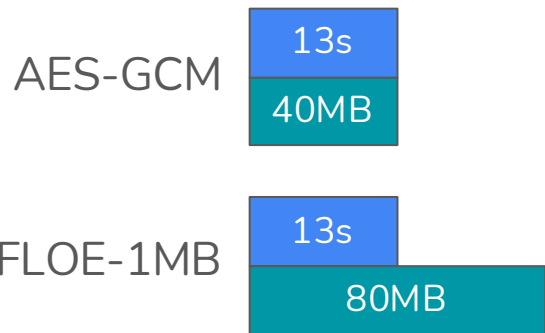


MacBook Pro, M3 Max, Java, 1GB

FLOE Performance

FLOE versus AES-GCM

“Streaming” Encryption

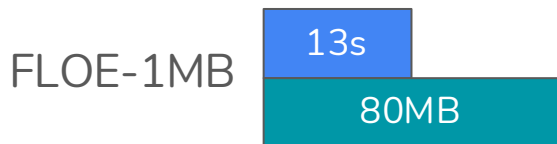


MacBook Pro, M3 Max, Java, 1GB

FLOE Performance

FLOE versus AES-GCM

“Streaming” Encryption



“Streaming” Decryption

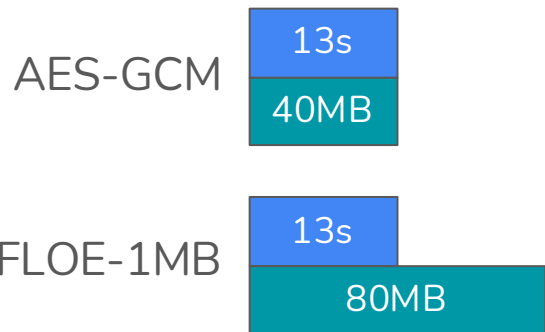


MacBook Pro, M3 Max, Java, 1GB

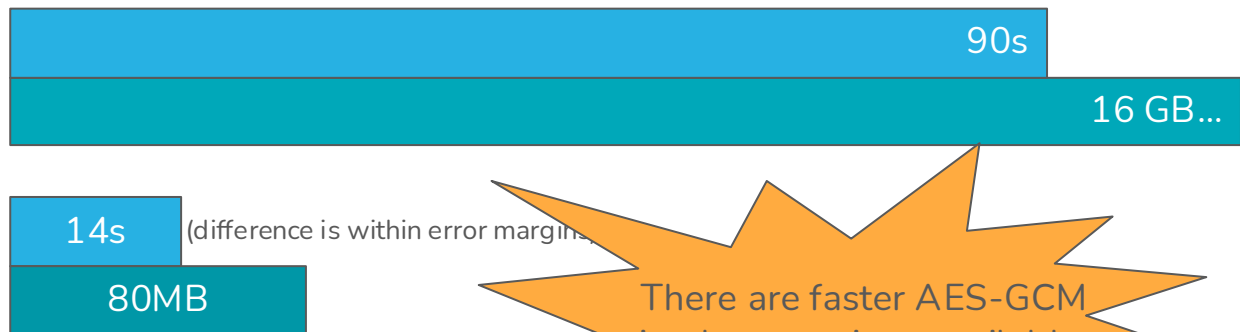
FLOE Performance

FLOE versus AES-GCM

“Streaming” Encryption



“Streaming” Decryption



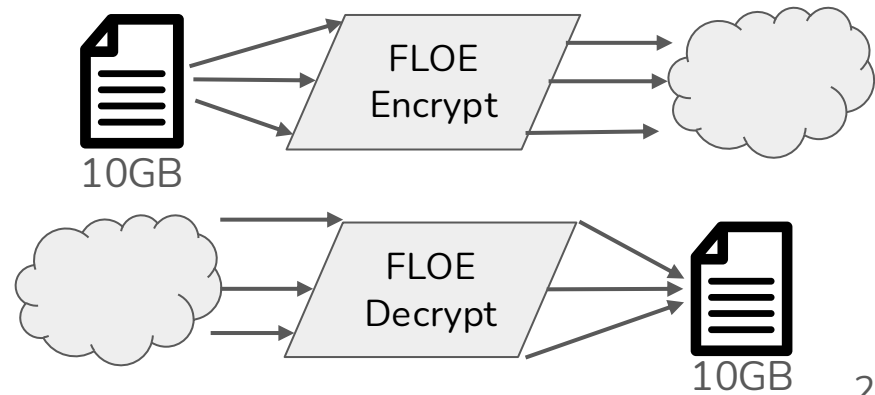
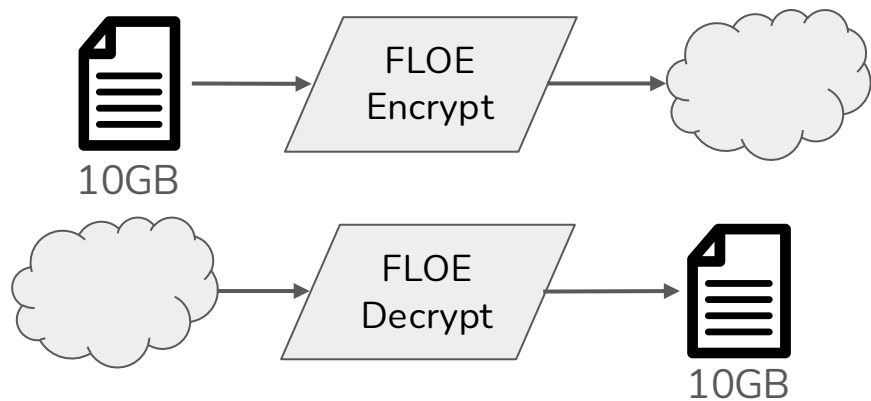
There are faster AES-GCM implementations available.



MacBook Pro, M3 Max, Java, 1GB

FLOE Performance

Sequential vs Multi-Threaded

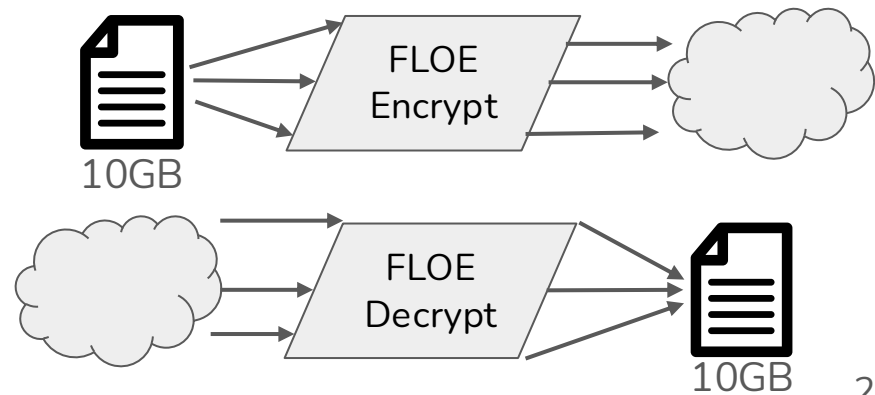
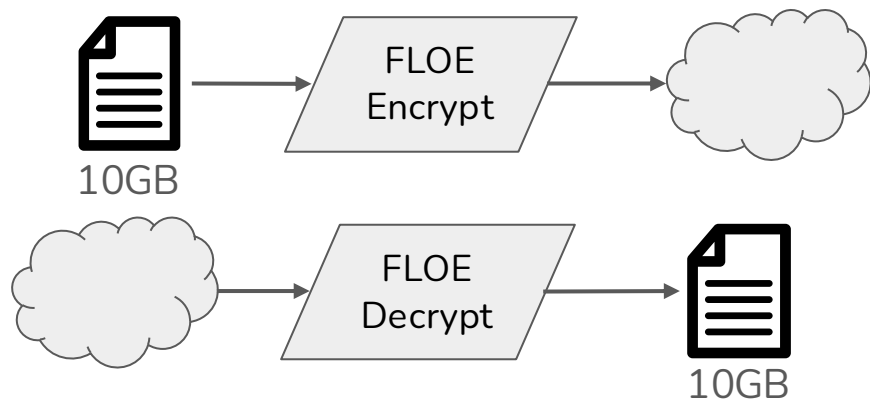


FLOE Performance

Sequential vs Multi-Threaded

Sequential

~ 3.5 minutes



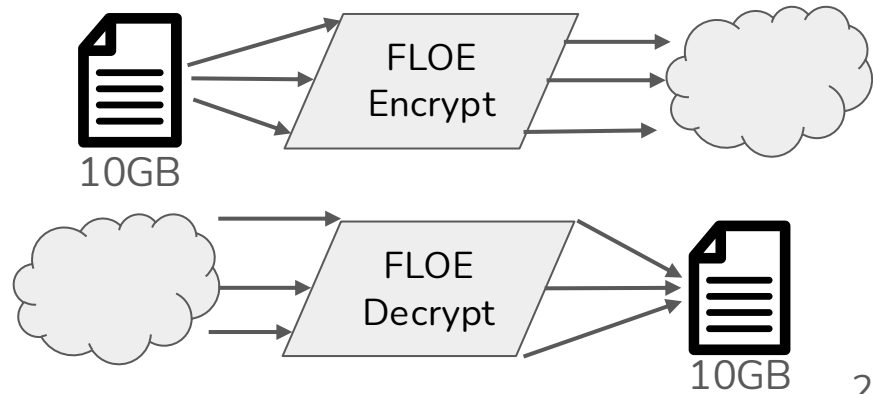
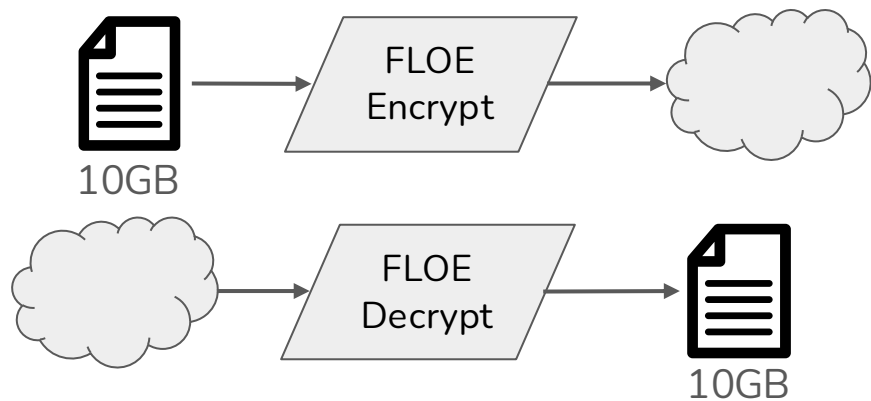
FLOE Performance

Sequential vs Multi-Threaded

Sequential

~ 3.5 minutes

CPU-bound



FLOE Performance

Sequential vs Multi-Threaded

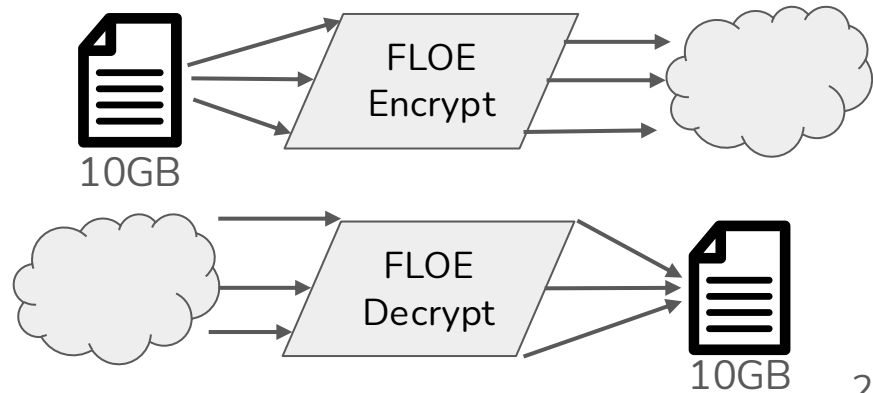
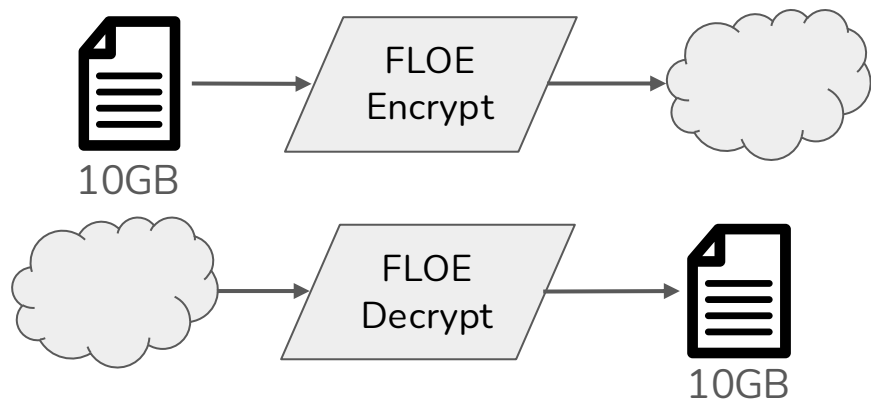
Sequential

~ 3.5 minutes

CPU-bound

Random-Access
Multi-threaded

~1.5 minutes



FLOE Performance

Sequential vs Multi-Threaded

Sequential

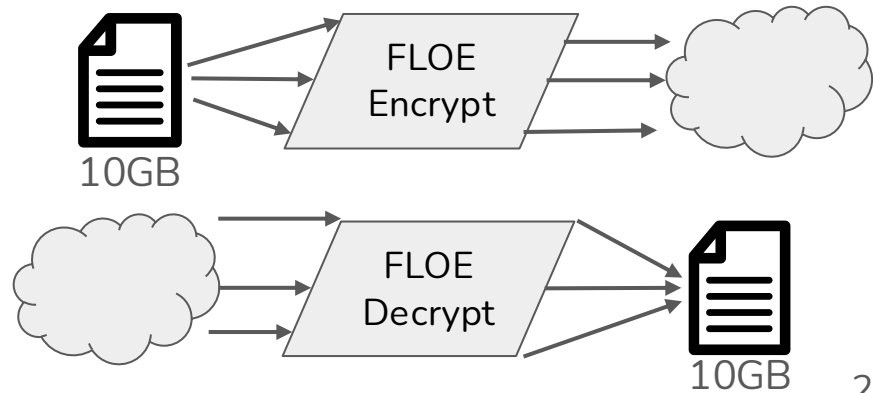
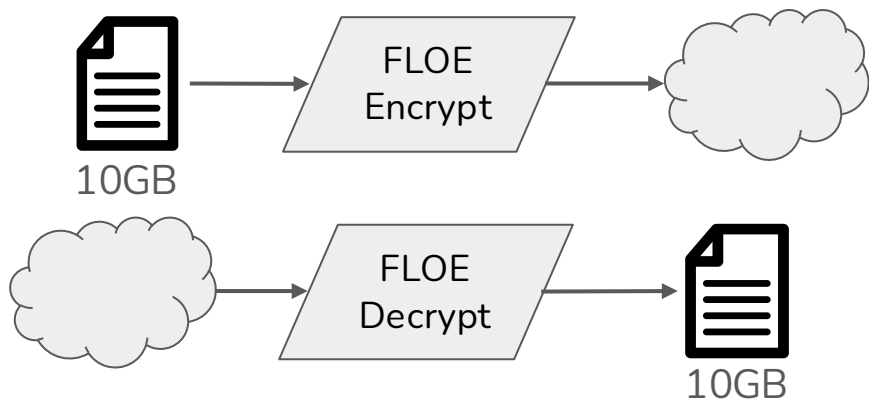
~ 3.5 minutes

CPU-bound

Random-Access
Multi-threaded

~1.5 minutes

Network-bound, 1Gbps



FLOE Performance

Sequential vs Multi-Threaded

Sequential

~ 3.5 minutes

CPU

Random-Access
Multi-threaded

~1.5 minutes

Network-bound, 1Gbps

Faster
Lower-Resources
More Secure



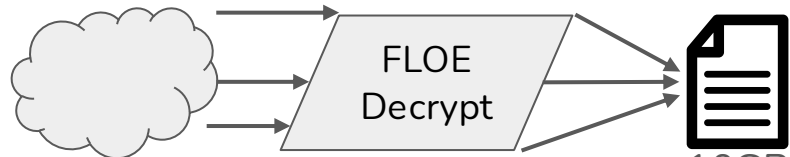
10GB



10GB



10GB



10GB

You can use FLOE now!

Public Specification

Available on GitHub

No patents

Anyone is allowed to use it

Reference Code

Available on GitHub

Apache 2.0 Licensed

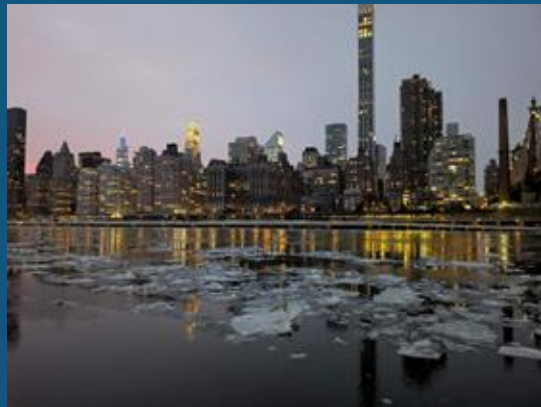
Java, Go, C++, and Rust

Community Involvement

Part of the Community
Cryptography Specification
Project (C2SP)

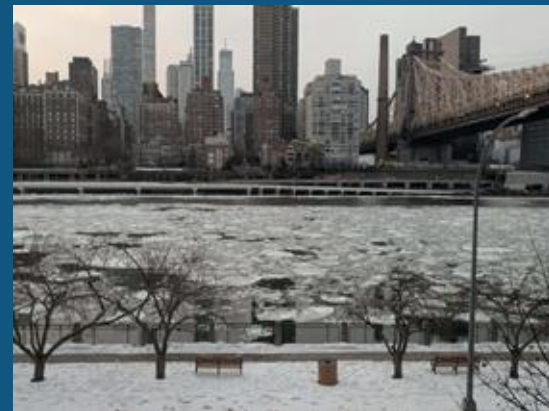
Already ported to the
language Zig in < 400
lines of code.

<https://github.com/Snowflake-Labs/floe-specification>



Thank you!

Questions?
jlen@cs.unc.edu



Paper



Reference implementations



See our paper for more details:

- Concrete bounds
- Security under multiple error messages
- Separations between definitions
- And much more!

<https://eprint.iacr.org/2025/2275>